University of Cabuyao
(PAMANTASAN NG CABUYAO)

COLLEGE OF COMPUTING STUDIES

UNSUPERVISED MACHINE LEARNING FOR ANOMALY-BASED DDOS MITIGATION IN SOFTWARE-DEFINED NETWORKS

A Thesis Proposal
Submitted to the Faculty of
The College of Computing Studies
PAMANTASAN NG CABUYAO
City of Cabuyao, Laguna

In Partial Fulfillment
of the Requirements for the Degree:
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

By:

Apellido, Jude Genesis T.
Monis, Andrea Pauline L.
Pacilona, Ramon Carlos E.
Prudente, Kin Edrian G.

May 2026

CHAPTER I

THE PROBLEM AND ITS SETTING

This chapter discusses the context and rationale for utilizing unsupervised machine learning to autonomously identify and mitigate Distributed Denial-of-Service (DDoS) attacks within Software-Defined Networks (SDN). It outlines the scope and delimitations, significance of the study, and the problem statement.

Introduction

Traditional networks are static and based on the 7-layer OSI model, where the Data Link and Network layers enable networking functions. These networks comprise three planes: the management plane for monitoring, the control plane for device coordination, and the data plane for handling traffic. Network changes must be made manually, making administration labor-intensive. In contrast, software-defined networking introduces a modern architecture that distinguishes the control plane from the data plane. A centralized controller manages network devices and directs traffic, making the network programmable and easier to manage [1]. This shift allows administrators to control resources more efficiently and automate network operations.

At the core of this modern architecture is the centralized SDN controller, which serves as the intelligent hub of the network. Rather than requiring administrators to configure every hardware device separately, SDN brings centralized control via software that operates independently of the underlying hardware [2], [3]. This allows the controller to gain a global view of the topology and quickly direct routing devices on how to

forward data packets, thereby providing highly flexible traffic management for communication between information sources.

Although this centralized flexibility offers unparalleled traffic control, it directly introduces a critical single point of failure. The SDN controller is highly vulnerable to Distributed Denial-of-Service (DDoS) attacks that can easily overwhelm its processing capacity and paralyze the network [3], [4]. The devastating impact of this vulnerability is evident in recent cyber events, ranging from Ransom DDoS (RDDoS) attacks blasting enterprise data centers with 10 to 50 Gigabits per second (Gbps), to massive IoT swarms like the Aisuru botnet bombarding cloud infrastructures with hundreds of millions of requests per second [29]. If a volumetric attack of this scale targets an unprotected SDN environment, the controller's resources are instantaneously exhausted. Furthermore, because legacy firewalls perform inadequately when defending the SDN control plane [1], this architectural flaw necessitates a modern, automated approach to prevent controller resource exhaustion.

To address SDN vulnerabilities, there are various existing studies that utilize machine learning-based defense technologies have been proposed; however, it reveals a significant gap. According to a comparative study, the traditional supervised algorithms that have been frequently evaluated such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Naive Bayes often optimizing their hyperparameters with Bayesian techniques prove that it can maximize efficiency on detecting Distributed Denial of Service (DDoS) attacks and minimize carbon emissions [34]. Similarly, other research has utilized Decision Trees and Artificial Neural Networks (ANN) to address real time detection in 5G networks and to mitigate data imbalance in training datasets [30] [35]. For highly complex environments such as Software Defined Networks integrating in Internet of Things (SDN-IoT), researchers have implemented advanced deep learning architectures, including the Multi-Hidden Deep Belief Network (MHDBN-W), as well as hybrid models integrating Convolutional Neural Networks (CNNs), Principal Component Analysis (PCA), and Vision Transformers [14] [36].

Contemporary approaches have even explored combining blockchain technology with clustering-based unsupervised algorithms and multi-objective feature selection to optimize attack detection [13], [44]. However, despite achieving high accuracy, these complex deep learning and supervised models share a critical limitation: they are highly computationally intensive. Running such resource-heavy algorithms continuously on the SDN controller can inadvertently exacerbate the exact CPU exhaustion they are designed to prevent. Furthermore, existing research predominantly focuses on attack identification without providing comprehensive, closed-loop response protocols. Any latency between detection and mitigation allows malicious traffic to persist, rapidly consuming vital network resources and ultimately overwhelming the controller.

Consequently, to protect the resource-sensitive SDN control plane, lightweight and highly efficient machine learning techniques, particularly unsupervised network anomaly detection methods remain the most viable solution for real-time monitoring [10]. To fulfill this requirement, this study deploys the Isolation Forest algorithm as the primary frontline defense. The utilization of unsupervised learning has proven highly effective for dynamic DDoS mitigation in large-scale environments [7]. Isolation Forest offers robust capabilities by mathematically identifying abnormal traffic patterns without requiring pre-labeled datasets. One of its primary advantages is the autonomous ability to instantly detect and quarantine previously unknown "zero-day" attacks, which is essential given the continuous evolution of adversarial tactics [46].

Once a potential threat is detected and quarantined by the Isolation Forest, a robust supervised method is required to accurately categorize the specific attack vector. To complement the initial unsupervised detection, this study utilizes the Random Forest algorithm as a secondary classification module. For instance, research by Ruikui Ma et al. [4] demonstrates that Random Forest yields exceptional efficacy in DDoS classification, achieving up to 99.99% accuracy in experimental evaluations. By constructing multiple decision trees to analyze distinct network traffic features and aggregating their predictions, Random Forest effectively processes complex traffic patterns while inherently mitigating overfitting. This sequential ensemble approach

enhances overall system reliability; it compensates for individual tree errors through collective decision-making, providing precise threat intelligence while ensuring the computationally heavier classification is strictly reserved for pre-filtered malicious traffic.

To address the inherent vulnerabilities of standalone models, combining multiple algorithms has consistently demonstrated superior performance [6]. By leveraging the unique strengths of individual models, hybrid frameworks can achieve remarkable accuracy while maintaining the low latency essential for real-time threat mitigation [7]. Building upon this premise, this study proposes an automated, anomaly-based mitigation system designed to secure the resource sensitive control plane of Software-Defined Networks.

The proposed system integrates this cascaded machine learning architecture: an unsupervised Isolation Forest for instantly detecting novel DDoS attack vectors without pre-labeled data, followed by a supervised Random Forest for accurate, high-speed attack classification. The anticipated outcome is a high-performance, resilient alternative to traditional signature-based systems, enabling the SDN controller to not only detect but autonomously mitigate malicious network flows in real-time.

Statement of the Problem

The general problem of this study is that Software-Defined Networking (SDN) architectures, while providing centralized network management, introduce a critical vulnerability to volumetric Distributed Denial-of-Service (DDoS) attacks. Existing security frameworks face a severe trade-off between detection accuracy and computational overhead, frequently resulting in high false-positive rates during traffic surges or causing controller CPU exhaustion due to resource-heavy analysis.

To resolve these issues, this study proposes a dual-layer machine learning framework integrated into the Ryu SDN controller to provide automated, resource-efficient DDoS mitigation.

This study aims to address these gaps in terms of:

1. How accurately can the Isolation Forest supported by the Random Forest algorithm detect DDoS attack patterns in terms of:

    1.1. false positive rate;

    1.2. true positive rate;

    1.3. false negative rate; and

    1.4 overall precision ?

2. How effectively does the lightweight Unsupervised Machine Learning driven system detect and mitigate an active attack by automatically implementing Flow Modification (FlowMod) blocking rules within the Ryu SDN controller in terms of:

    3.1 detection time;

    3.2 mitigation response time;

    3.3 controller resource overhead; and

    3.4 autonomous execution reliability ?

3. What are the users' assessments of the proposed mitigation system based on the following ISO/IEC 25010 criteria:

    3.1. functional suitability;

3.2. performance efficiency; and

3.3. operability?

4. What is the level of acceptability of the proposed mitigation system as assessed by IT Experts' based on the ISO/IEC 25010 standards in terms of:

4.1. functional suitability;

4.2. performance efficiency;

4.3. operability;

4.4. reliability;

4.5. maintainability; and

4.6 resistance ?

Scope and Limitation

This study encompasses the design, simulation, and performance evaluation of an automated, anomaly-based mitigation system utilizing a lightweight, cascaded machine learning architecture (Isolation Forest and Random Forest). The scope is explicitly focused on mitigating volumetric Distributed Denial-of-Service (DDoS) attacks within a Single-Controller Software-Defined Networking (SDN) framework.

To ensure an isolated and controlled evaluation without risking physical infrastructure, the technical implementation is bounded within a virtualized sandbox environment. The simulation is strictly limited to an Ubuntu 22.04 LTS operating system, utilizing the Mininet network emulator to orchestrate a custom tree topology, and the

Ryu SDN Controller for centralized management via the OpenFlow v1.3 protocol. Traffic generation is scoped to simulated malicious DDoS floods and simulated benign background traffic; no live human user traffic or physical hardware switches will be utilized.

To maintain a lightweight profile and prevent controller saturation, the system's operational scope is divided into four strictly defined functional modules:

First is the **Traffic Extraction Module.**The scope of network visibility is limited strictly to polling OpenFlow switches for Layer-3 and Layer-4 header metadata, specifically packet counts, byte rates, and flow durations. To preserve computational resources and user privacy, this study does not encompass Deep Packet Inspection (DPI) or the examination of encrypted packet payloads.

Detection and Classification Module. The threat detection boundary is confined to the cascaded deployment of the Isolation Forest for anomaly detection to isolate malicious network anomalies and the Random Forest for the classification of the attacks. The training and validation phases are exclusively scoped to the Simulated Network Traffic (SNT) Dataset (2025) and the DDoS Attacks (2023) Dataset from Mendeley Data to establish a standardized benchmark. The system is trained solely to identify volumetric DDoS attacks and does not cover other cyber threats such as malware, phishing, or SQL injections.

Automated Mitigation Module. The scope of the response protocol is restricted to generating instantaneous Flow Modification (FlowMod) directives. Upon classification of a threat, the controller is limited to executing switch-level blocking rules (e.g., dropping packets from malicious IPs or ports) and does not extend to complex firewall routing or external traffic scrubbing services.

Finally, in the **Performance Monitoring and Visualization Module**, the evaluation of the system is strictly confined to quantifiable performance metrics, specifically detection accuracy, false-positive rates, mitigation latency, and CPU or

memory utilization. This module translates simulated network events into dynamic graphical visualizations, limiting its scope to visually tracking the system's ability to preserve legitimate network throughput during an active attack simulation.

Beyond the virtual sandbox, the scope of this study includes a qualitative technical validation phase. To assess the practical applicability and operational viability of the proposed framework, the system's architecture and simulation results will be evaluated by active network administrators or engineers operating within Laguna, Philippines. This validation will be conducted through semi-structured interviews, allowing the researchers to gather real world insights on how the proposed Software Defined Network mitigation strategy aligns with local enterprise infrastructure capabilities and current industry standards.

Despite its automated capabilities, the study is subject to several delimitations and operational constraints.

To maintain a lightweight profile and prevent controller saturation, the framework does not utilize Deep Packet Inspection (DPI) or examine encrypted payloads. It relies exclusively on Network and Transport layer (Layer 3 and Layer 4) flow characteristics, such as packet counts, byte rates, and flow durations. Consequently, the automated Flow Modification (FlowMod) response is specifically optimized for volumetric attacks and is not designed to mitigate Application Layer (Layer 7) attacks, such as HTTP GET floods, which closely mimic legitimate web requests and require proxy level filtering beyond basic OpenFlow switch capabilities.

Furthermore, while the unsupervised model seeks to identify zero-day anomalies, the baseline training is conducted entirely using the SNT (2025) and DDoS Attacks (2023) datasets. Without continuous real-time learning on live data, the model may not accurately represent novel botnet architectures or attack vectors developed after the publication of these datasets.

Additionally, because the evaluation relies exclusively on simulated benign traffic generated within the Mininet sandbox, the study cannot fully capture the unpredictable, highly variable behavioral nuances of genuine human users operating diverse physical devices on a live enterprise network.

Finally, the system's mitigation capacity is strictly constrained by the processing and memory limitations of the host computer. While the system is capable of identifying large-scale attacks, total mitigation cannot be ensured if the traffic volume surpasses the virtual machine's processing capabilities. Defending against massive volumetric floods demands significant computational power; thus, the virtual environment may crash if attack volumes exceed this limit before the controller is able to effectively implement blocking rules. Therefore, payload-level inspection, multi-controller or decentralized networks, physical hardware implementations, and commercial enterprise routers are excluded from this study and deferred for future research.

Significance of the Study

The findings of this study are highly significant as they address the critical need for accessible and automated cybersecurity within Software-Defined Networks (SDN). By developing a resource-efficient DDoS mitigation framework using a cascaded machine learning architecture, this research provides a cost-effective alternative to proprietary systems and significantly reduces administrative overhead through automated threat response. Specifically, the proposed system and the results of this study will be highly beneficial to the following:

Corporate Enterprises and Industrial Networks: Corporate networks frequently manage massive, unpredictable traffic surges and require strict data availability. This research offers industrial enterprises a highly scalable security model that protects their logically centralized infrastructure from destructive volumetric attacks. By automating the threat response, the framework secures business-critical applications and

communications without degrading legitimate employee network access or requiring expensive proprietary hardware upgrades.

Network Administrators. The study addresses operational challenges faced by network administrators, including alert fatigue from false positives and delays in manual threat response. The automated detection and mitigation system reduces reliance on continuous manual traffic analysis by identifying and blocking DDoS flows through the SDN controller. This allows security personnel to prioritize strategic security planning over reactive incident response. Furthermore, integrating Isolation Forest for anomaly detection reduces false alarms by distinguishing legitimate traffic spikes from actual attacks.

The Researchers. This study provides the researcher with experience in implementing machine learning algorithms within SDN environments. By developing and testing the automated mitigation system, the researcher gains experience with Random Forest and Isolation Forest applications, OpenFlow protocol implementation, and the challenges of balancing detection accuracy with response latency.

Future Researchers. This study documents the implementation of machine learning for DDoS mitigation in SDN environments. Future researchers can use this study as a baseline for comparative studies, such as testing alternative algorithms, evaluating performance across different attack types, or extending the system to support distributed SDN controllers.

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents various foreign and local literature focusing on unsupervised machine learning techniques and anomaly-based DDoS mitigation strategies within Software-Defined Networks (SDN). The gathered information includes books, journals, articles, and related studies from the internet that provide the theoretical foundation for the proposed system.

Conceptual Literature

Standard Features

A fundamental concept in SDN is its layered architecture, which includes the application layer, control layer, and infrastructure layer. Communication between these layers is facilitated by standardized protocols such as OpenFlow, which enables the controller to dynamically manage traffic flows across network devices [62]. This programmability allows for real-time monitoring and rapid response to network anomalies. Another essential concept is network virtualization, which enables the creation of simulated network environments such as High-Fidelity Virtualized Testbeds for testing and evaluation without affecting physical infrastructure [20], [21]. Virtualized environments are widely used in cybersecurity research to safely analyze attack behaviors and validate defense mechanisms.

In addition, traffic analysis plays a crucial role in network security. Instead of relying on resource-intensive techniques such as Deep Packet Inspection (DPI), modern conceptual approaches utilize passive flow-based feature extraction [44]. This method

analyzes statistical attributes such as packet counts, byte rates, and flow durations, which are sufficient for identifying abnormal traffic patterns while maintaining system efficiency. To act upon this data, standard frameworks incorporate automated mitigation mechanisms that dynamically generate flow rules to block malicious traffic at the network level [43].

Guidelines for Using the Algorithms

Machine learning has become a key approach in intrusion detection systems due to its ability to identify patterns and anomalies in large-scale network data [45]. One widely used unsupervised learning algorithm is the Isolation Forest, which is designed to detect anomalies by isolating data points that significantly differ from normal behavior [60]. The algorithm operates on the principle that anomalies are “few and different,” making them easier to isolate compared to normal instances. According to Lakshmi et al. [10], the Isolation Forest algorithm demonstrates high effectiveness in detecting anomalous patterns in SDN environments without requiring labeled datasets, making it suitable for dynamic and evolving network conditions.

Complementing this approach is the Random Forest algorithm [63], a supervised learning technique that constructs multiple decision trees and aggregates their outputs to improve classification accuracy. This ensemble method is known for its robustness and ability to handle high-dimensional data. As demonstrated by Chu et al. [5], Random Forest models achieve high accuracy in classifying DDoS attack types, making them reliable for intrusion detection tasks. By combining unsupervised and supervised learning techniques through hybrid detection systems, where anomaly detection mechanisms trigger secondary classification models, overall detection and classification performance can be improved while optimizing computational resources.

User-Centric Design

User-centric design is an essential concept in system development that focuses on enhancing usability and user experience [18]. In network security systems, administrators require clear and accessible interfaces to monitor network conditions and respond to threats effectively. Principles such as visual clarity, real-time feedback, and intuitive navigation are critical in designing monitoring dashboards.

Data visualization techniques, including charts and graphs, play a significant role in presenting complex network data in an understandable format. Real-time dashboards enable administrators to quickly identify anomalies, assess system performance through Key Performance Indicators (KPIs), and make informed decisions. Furthermore, conceptual literature emphasizes the necessity of logging and analytics mechanisms that provide administrators with detailed records of automated system actions, logging and analytics mechanisms that provide administrators with detailed historical records of automated system actions. By applying human-computer interaction (HCI) principles, systems can reduce cognitive load, improve operational efficiency, and ensure the administrator remains the final operational authority.

Testing Tools and Procedure

In cybersecurity research, controlled testing environments are essential for evaluating system performance without risking real-world infrastructure [22]. Virtualized network environments allow researchers to simulate realistic traffic scenarios, including both normal and malicious activities. This enables the safe analysis of attack patterns and the validation of detection mechanisms across various simulated network scenarios [42].

Datasets specifically designed for SDN environments are commonly used to evaluate machine learning models [37]. These datasets contain labeled instances of both normal traffic and various attack types, allowing researchers to measure performance metrics such as accuracy, true positive rate, and false positive rate. According to existing studies, evaluating models using multiple datasets evaluating models using multiple benchmark datasets improves reliability and ensures that the system can generalize across different network conditions. improves reliability and ensures that the system can generalize across different network conditions.

Performance evaluation in intrusion detection systems typically involves metrics such as detection accuracy, false positive rate, and response time (mitigation latency). These metrics provide insight into the effectiveness and efficiency of the system in identifying and mitigating threats. Continuous monitoring and evaluation, including strict observation of the monitoring system resource utilization, are necessary to ensure that detection mechanisms remain lightweight and effective against evolving cyber threats [20].

Research Literature

This section presents relevant foreign and local literature, including journals, books, and prior studies, that support the theoretical foundation of the proposed anomaly-based DDoS mitigation approach in Software-Defined Networks (SDN). It discusses key issues in SDN security, particularly vulnerabilities in centralized control planes, and highlights the transition from traditional signature-based methods to machine learning-based detection techniques. Furthermore, it examines existing approaches, their limitations, and research gaps related to computational efficiency, real-time response, and handling high-volume network traffic.

Review of Related Literature

Vulnerabilities in Software-Defined Networking (SDN) Architecture

Software-Defined Network (SDN) architecture offers numerous advantages such as increased flexibility, scalability, and improved network management but also introduces new and notable security challenges such as Distributed Denial-of-Service (DDoS) attacks. These attacks aim to overwhelm targets with malicious traffic and even short-lived DDoS incidents can drastically impact the entire network's stability, performance and availability [1]. According to Najafimehr et al. [8], modern DDoS attacks specifically target the SDN controller as a "single point of failure," where a successful flood can incapacitate the entire network.

This architectural vulnerability is fundamentally tied to the control plane's processing capacity. When centralized controllers are required to inspect large volumes of traffic during high-speed attacks, computational load increases significantly, often resulting in resource exhaustion and performance degradation [17], [19].

Furthermore, comparative performance assessments confirm these natural hardware limitations, demonstrating that logically centralized controllers are highly prone to crashing when subjected to the dual strain of processing volumetric malicious traffic and maintaining normal network operations simultaneously [20]. In fact, recent vulnerability analyses demonstrate that even distributed SDN controllers operating within robust cloud environments remain highly susceptible to resource exhaustion under volumetric floods [3]. Consequently, minimizing operational overhead remains a critical priority even in robust enterprise environments, where optimizing computational resources and preventing controller saturation is paramount for maintaining high-speed network availability [18].

As botnets grow more advanced by utilizing hijacked IoT devices to execute attacks the centralized "brain" of the network requires a specialized defense layer operating directly within the control plane [14]. The urgency for automated defense mechanisms is underscored by recent industry threat reports indicating that 89% of modern network-layer attacks conclude in under 10 minutes [31]. This suggests that manual mitigation may not respond quickly enough to protect the controller from resource exhaustion, making automated, machine learning-driven mitigation increasingly necessary.

Limitations of Traditional Defenses and the Shift to Machine Learning

Traditional network security has long relied on signature-based detection, which functions by matching incoming traffic patterns against a static database of known threats. However, recent studies emphasize that this approach is fundamentally incapable of identifying "Zero-Day" or rapidly evolving Distributed Denial of Service Attacks [66]. A study by S. Peng et al. [13] stated that traditional networking relies heavily on rigid, hardware-based firewalls; these legacy systems often struggle to handle unexpected volumetric bursts, leading to service degradation for legitimate users. Furthermore, R. Chu et al. [41] highlight that static models that wait for known signatures are ineffective against new attack vectors since botnet behaviours are continually changing, this phenomenon is known as "concept drift."

Consequently, researchers advocate for a decisive shift toward Anomaly Detection Theory, which focuses on establishing a behavioral baseline of typical network activity to identify statistical outliers [55]. Unsupervised learning algorithms are particularly effective for this, as they do not require labeled training data to function. Pattanayak et al. [11] evaluated various unsupervised techniques and concluded that the Isolation Forest algorithm provides an optimal balance between anomaly detection capability and computational efficiency for intercepting zero-day attacks.

By evaluating complex network environments, such as those flooded by malicious IoT devices, studies have proven that unsupervised models can successfully identify anomalous behaviors entirely by bypassing the need for labeled attack signatures [15]. By removing the dependency on pre-existing signatures, defensive frameworks gain the operational advantage of detecting entirely new, unseen volumetric floods purely through their mathematical deviation from normal traffic.

However, while anomaly detection is incredibly fast and effective at catching unknown threats, it introduces distinct operational challenges if deployed as a standalone defense. Lakshmi et al. [10] systematically evaluated the Isolation Forest method within the Software Defined Network (SDN) security and revealed that relying on it as the sole decision-making application strictly limits overall precision and recall. Because unsupervised models identify statistical deviations rather than classifying specific attack typologies, they generally suffer from higher false-positive rates compared to supervised methods. In a live enterprise, this imprecision creates a severe operational risk, as the system may accidentally block legitimate users during natural surges in traffic.

Furthermore, while the shift to machine learning is necessary, a significant gap remains in how these anomaly detection models are trained. A substantial portion of current SDN research, for instance a study by Shaikh et al. [14], still relies on the CIC-DDoS2019 dataset. While comprehensive for general web-based attacks, this dataset was captured in traditional network environments and fails to account for the unique operational behaviors of an SDN architecture, such as flow-table overflow, control-plane communication latency, and OpenFlow-specific telemetry [37]. Similarly, recent deep learning approaches [36] often utilize generalized traffic captures, or IoT-centric data such as the Kitsune dataset utilized by Ram et al. [15], which do not reflect the resource-constrained environment of a live centralized controller.

This reliance on outdated, non-SDN specific data creates a "training-to-reality" mismatch, where models may achieve high accuracy on static web traffic but struggle to

handle the dynamic pressures of a real world network. Consequently, when local network administrators attempt to deploy these theoretical models to protect real-world enterprise architectures, the systems often fail or produce high false-positive rates. This operational failure highlights the critical need for frameworks trained on modern, SDN-specific datasets.

Computational Efficiency of Architectures in SDN Defense

As Software-Defined Networks scale to accommodate high-speed data planes, the computational cost of security mechanisms becomes a critical operational constraint. While deep learning offers high detection rates, deploying heavy, single-stage architectures continuously on a live centralized controller risks rapidly exhausting its CPU resources during a high-speed attack [25]. For instance, complex models integrating Convolutional Neural Networks and Vision Transformers, as proposed by Shaikh et al. [14], or hybrid deep learning approaches utilized by Alsufyani et al. [36], require immense processing power to function.

Furthermore, relying on dynamic, continuous model-searching loops, such as Automated Machine Learning (AutoML) frameworks [26], introduces massive computational overhead that can lead to network degradation under flood conditions. If a security mechanism is too computationally demanding, it inadvertently aids the attacker by turning the controller itself into a bottleneck, effectively executing the Denial-of-Service on the network's behalf.

To address this CPU limitation, researchers have explored methods to physically relieve the central controller. For example, Ma et al. [4] suggest pushing classification algorithms directly to the edge switches, while Roopak et al. [44] utilize multi-objective feature selection to minimize controller latency by drastically reducing the total number of dataset features processed. Additionally, frameworks by Ko et al. [7] leverage

lightweight, unsupervised learning equipped with hierarchical feature selection to completely bypass the heavy processing overhead associated with labeled datasets.

By stripping away complex verification layers and reducing the data footprint, these systems successfully preserve controller CPU; however, they inherently sacrifice the deep analytical rigor required to accurately distinguish between a malicious botnet flood and a legitimate surge in enterprise network traffic. Consequently, these lightweight frameworks often suffer from a severe "precision-speed trade-off."

Surveys of machine learning-based detection methods highlight this exact limitation: while unsupervised anomaly detection is lightweight and excels at catching unknown zero-day attacks, it suffers from significantly higher false-positive rates. For instance, Lakshmi et al. [10] evaluated the Isolation Forest algorithm as a standalone, single-layer security application and revealed strict limitations in overall precision and recall. Conversely, highly accurate hybrid models such as those stacking multiple heavy algorithms provide near-perfect accuracy but require enterprise-grade computational hardware that is far too expensive and heavy for standard SDN controllers [6], [24]. This highlights the critical necessity for a highly optimized, cascaded architecture that can balance the speed of unsupervised filtering with the accuracy of supervised classification.

Hybrid and Cascaded Approaches in SDN Defense

To address limitations of single-model approaches that struggle to balance computational efficiency with detection accuracy, securing resource-constrained SDN environments requires a structural compromise. Deploying heavy supervised models across all network traffic guarantees controller saturation, while relying solely on lightweight unsupervised models introduces unacceptable false-positive rates [10]. Consequently, recent advancements in machine learning emphasize the theoretical necessity of hybrid, multi-tier architectures. Recent studies aim to combine the speed of

unsupervised filtering with the accuracy of supervised classification without inheriting their respective hardware costs [45].

Within this context, literature frequently evaluates specific algorithms for their hardware efficiency. For instance, unsupervised algorithms like the Isolation Forest are mathematically noted for their low computational complexity, rapidly processing OpenFlow telemetry to preserve controller CPU [60]. However, studies confirm that their standalone deployment lacks the precision required for enterprise defense. Conversely, supervised algorithms like the Random Forest classifier are heavily cited for their high accuracy and ability to reduce false positives during DDoS attacks [5], [23]. Yet, the existing literature highlights a persistent bottleneck: deploying these supervised models continuously on all network traffic remains too heavy for standard controllers. Therefore, a major architectural gap remains in balancing detection accuracy with computational efficiency in SDN environments.

Furthermore, beyond these algorithmic trade-offs, a critical gap persists in how these defenses are operationalized. Current SDN security literature overwhelmingly focuses on attack detection, identifying anomalies and alerting an administrator rather than active, closed-loop prevention. For a lightweight architecture to be truly viable in a centralized network, researchers emphasize the need for autonomous mitigation protocols that execute before a switch's memory capacity is exceeded [37]. While the OpenFlow protocol theoretically allows controllers to push reactive Flow Modification (FlowMod) rules to block threats at the data plane [56], [62], there remains a significant lack of empirical research demonstrating the seamless, automated integration of lightweight machine learning models directly with OpenFlow mitigation execution.

SDN Emulation and Controller Performance

To safely evaluate proposed security architectures, utilizing high-fidelity emulation tools has become a critical methodological standard for validating frameworks

without the prohibitive costs and operational risks associated with physical hardware deployment.

Borgianni et al. provide empirical validation for this approach by comparing network emulation against physical testbeds, demonstrating that Mininet offers a high degree of accuracy for distributed systems research while successfully mirroring physical hardware performance up to the host system's natural limits [21]. This is further supported by Almaini et al., who argue that establishing comprehensive real-world SDN environments is often financially unfeasible and physically fragile; consequently, software-based simulations serve as the standard prerequisite for safely evaluating network solutions prior to any real-world implementation [22]. The versatility and reliability of this emulator are heavily documented, with researchers utilizing it to build highly complex, customized simulation environments that accurately reflect real-world infrastructure [42].

Beyond general network simulation, Mininet has proven exceptionally reliable for evaluating specific SDN controller constraints. Nowsin et al. conducted a comprehensive comparative assessment of logically centralized controllers, utilizing Mininet to effectively measure how controllers like Ryu handle varying traffic loads and CPU utilization under acute stress [20]. Furthermore, this environment is crucial for observing the actual latency of rule deployments, such as the delay in Flow Modification (FlowMod) execution from the controller to the data plane during an active attack [43].

To address the rigorous requirements of active security research, multiple studies including Karthika and Arockiasamy [16] and Alashhab et al. [12] have successfully utilized this exact simulated environment to deploy machine learning models, extract OpenFlow telemetry, and trigger automated mitigation strategies directly within the control plane. This mirrors the exact architectural setup used to generate modern, high-fidelity SDN datasets such as Simulated Network Traffic Dataset (2025) [67] and DDoS Attacks (2023) datasets [66] by Mendeley, ensuring complete

methodological consistency between the AI's training data and the live evaluation environment.

Consequently, the reliance on high-fidelity emulators is not merely a compromise for hardware constraints, but a scientifically validated necessity for modern cybersecurity research. By isolating the SDN topology within a virtualized emulator, researchers can safely generate extreme, destructive botnet traffic to push a centralized controller's CPU to its absolute limit. This methodology provides a rigorous, zero-risk environment to evaluate the performance and efficiency of machine learning-based detection frameworks under controlled conditions without risking physical hardware failure or triggering a real-world enterprise or campus network outage.

Review of Related Studies

Performance and Constraints of Supervised Classifiers

Current literature heavily relies on supervised machine learning models to secure SDN environments, often prioritizing classification accuracy above all other metrics. Studies by T. S. Chu et al. [5] and Haque et al. [27] frequently showcase the Random Forest algorithm as the premier choice, demonstrating near-perfect detection rates in controlled experimental evaluations. Ibrahim et al. [33] further cement this trend by utilizing feature-optimized machine learning, proving that Random Forest excels at handling high-dimensional traffic features to achieve high-fidelity classification.

However, a critical review of these implementations reveals a severe operational gap: the assumption of infinite controller resources and static threat landscapes. While the methodologies proposed by Chu [5] and Ibrahim [33] achieve remarkable accuracy on known datasets, their continuous deployment of supervised classifiers introduces a fatal processing bottleneck in live SDN environments. As Fatiyah et al. [35] point out, extracting high-dimensional features requires the SDN controller to deeply inspect

continuous streams of packet headers. In the event of a high-volume volumetric attack, this continuous, heavy feature extraction will exhaust the controller's CPU and memory resources long before the Random Forest model even completes its threat inference. By functioning as "always-on" sentries, these heavily supervised architectures paradoxically become the very bottleneck that causes the network to crash.

Furthermore, the rigid reliance on supervised models creates a definitive architectural blind spot. Because these models are intrinsically bound to the specific attack signatures they were trained on, they are fundamentally incapable of recognizing "Zero-Day" anomalies undocumented, novel threats for which no training data exists [47]. Consequently, the defensive frameworks proposed in recent literature remain highly vulnerable to the rapidly evolving tactics of modern, AI-driven botnets.

To bridge this survivability and adaptability gap, the current research proposes a fundamental architectural shift. Instead of deploying the resource-intensive Random Forest classifier as an always on, continuous monitor which leads to the controller exhaustion seen in previous models, this study embeds it within a cascaded pipeline. Random Forest is removed from the initial monitoring path and is triggered only as a second-tier verifier after a lightweight, unsupervised process (Isolation Forest) flags a potential zero-day anomaly. This specific architectural intervention directly resolves the limitations of the current literature: it guarantees zero-day detection capabilities while entirely preserving the controller's CPU resources during periods of normal traffic, ensuring the network survives the initial flood of an attack.

Architectural Resilience and Real-time Mitigation

A primary focus of contemporary research is the vulnerability of the centralized SDN controller as a single point of failure. Ma et al. [4] and Horsanali et al. [26] investigated adaptive response mechanisms to maintain network stability, with Ma et al. [4] specifically focusing on real-time detection to minimize the window of vulnerability.

Despite these advancements, Najafimehr et al. [8] note that the success of mitigation is often dictated by the "attack magnitude," which can still overwhelm a controller if the detection logic is too resource-intensive.

This architectural strain is further analyzed by Zaheed et al. [38], who surveyed the inherent insecurities of the SDN control plane, noting that many existing frameworks prioritize "detection accuracy" over "mitigation latency." This creates a dangerous gap where malicious traffic continues to saturate the network while the system processes the threat. Sanjeetha et al. [17] argue that a mitigation protocol must be lightweight enough to trigger in milliseconds to prevent the switch-to-controller channel from becoming a self-inflicted bottleneck during high-volume floods.

The current research addresses these findings by prioritizing mitigation speed through a cascaded architecture. By utilizing a lightweight first-layer filter based on unsupervised methods, the system initiates a response faster than single-stage deep learning models. This approach ensures the controller remains resilient even under high-volume volumetric floods, directly solving the "decision lag".

Evolution Toward Unsupervised and Hybrid Models

To address the limitations of supervised learning, researchers have increasingly turned toward unsupervised anomaly detection. Peng et al. [2] proposed a defense strategy utilizing unsupervised learning to identify "zombie" traffic as statistical outliers, which does not require the pre-labeled datasets that supervised models depend on.

This shift is supported by Lakshmi et al. [10], who evaluated the Isolation Forest method specifically for SDN security, finding it highly effective at identifying anomalies in high-dimensional network data by measuring how easily a data point can be isolated from the rest of the flow.

Similarly, Ram et al. [15] demonstrated the utility of Isolation Forest for unsupervised detection in IoT-related attacks, proving its robustness against evolving threats. However, a recurring challenge noted by Shariq et al. [45] and Choi et al. [46] is the "precision-speed trade-off." While lightweight unsupervised models achieve the processing speeds necessary for real-time SDN operation, they may suffer from elevated false-positive rates during legitimate traffic surges, as the system might mistake a sudden burst of legitimate "flash crowd" traffic for a statistical anomaly.

The current research bridges this gap by implementing a hybrid cascaded approach. By placing the Isolation Forest as the primary filter for speed and the Random Forest as the validator for precision, the proposed system aims to achieve high-fidelity classification without the massive computational weight typically associated with simultaneous hybrid deployments [24]. This structure allows for the discovery of unknown threats through the unsupervised layer while maintaining the analytical rigor of the supervised layer.

Bridging the Decision to Action Protocol Deficiency

The efficacy of machine learning models in operational SDN environments is often undermined by the acute class imbalance between ubiquitous legitimate traffic and infrequent adversarial packets. This vulnerability is especially evident when supervised classifiers are trained and evaluated on standard, high-volume benchmark datasets.

Studies such as those by T. S. Chu et al. [5], Haque et al. [27], and Estupiñán Cuesta et al. [32] frequently rely on global benchmarks like the CICDDoS2019, NSL-KDD, and ISCX datasets, as well as specialized control-plane and IoT logs like the Kitsune Dataset [15]. Because these real-world and simulated datasets inherently contain a heavily skewed distribution of traffic classes, they introduce a significant point of failure for traditional supervised models.

The critical gap in these existing studies lies in their fundamental reliance on perfectly balanced, labeled data. To achieve the high accuracy rates reported in their experimental evaluations, researchers are often forced to apply computationally heavy data resampling techniques (such as oversampling minor classes or undersampling legitimate traffic) to artificially balance the datasets before training. However, in a live SDN control plane, network traffic cannot be artificially paused or resampled. If a supervised model is deployed in a real-world scenario without this artificial balancing, it inevitably suffers from severe class bias, defaulting to classifying almost all traffic as "normal" and failing to detect low-volume or zero-day attacks [47].

Furthermore, this reliance on historical dataset profiling creates a severe operational blind spot. By training exclusively on the known attack signatures present in datasets like CICDDoS2019 or the SNT logs, supervised classifiers become structurally incapable of recognizing undocumented anomalies. Treating anomaly detection purely as a multi-class categorization problem rather than a statistical outlier problem leaves the SDN highly vulnerable to novel, unrecorded botnet tactics.

The current study directly bridges this resilience gap by shifting the architectural paradigm away from data-dependent supervised classifiers. Instead of relying on perfectly balanced datasets, this research integrates an unsupervised algorithm (Isolation Forest) as the initial detection phase within the Ryu controller.

Because this mathematical approach treats anomalies purely as statistical outliers isolating "few and different" packets it completely bypasses the need for labeled attack signatures or real-time data resampling. By leveraging this unsupervised capability to handle the raw, imbalanced traffic first, the proposed cascaded system negates the class bias vulnerabilities prevalent in current literature, yielding a lightweight and scalable defensive mechanism.

This ensures that once a threat is validated by the cascaded pipeline, it autonomously isolates the malicious flows at the edge, preserving controller resources for legitimate traffic without requiring human oversight.

Addressing the Data Imbalance Resilience Gap

The efficacy of machine learning models in operational SDN environments is often undermined by the acute class imbalance between ubiquitous legitimate traffic and infrequent adversarial packets. This vulnerability is especially evident when supervised classifiers are trained and evaluated on standard, high-volume benchmark datasets. Studies such as those by T. S. Chu et al. [5], Haque et al. [27], and Estupiñán Cuesta et al. [32] frequently rely on global benchmarks like the CICDDoS2019, NSL-KDD, and ISCX datasets, as well as specialized control-plane and IoT logs like the Kitsune Dataset [15]. Because these real-world and simulated datasets inherently contain a heavily skewed distribution of traffic classes, traditional supervised models require extensive data resampling and profiling of "normal" behavior. If the data is not artificially balanced, these models suffer heavily from class bias.

Recent inquiries, such as the work by R. Chu et al. [41], address this dynamic by utilizing concept drift analysis based on the Isolation Forest algorithm. Unlike traditional models that struggle with the severe data imbalance found in benchmarks like CICDDoS2019 or the SNT dataset, the Isolation Forest is architecturally predicated on the rapid isolation of "few and different" anomalies rather than profiling normal classes [10].

Pattanayak et al. [11] further support this by concluding that the Isolation Forest provides a superior balance between anomaly detection capability and computational

efficiency, particularly for zero-day attacks that do not even exist in historical datasets like NSL-KDD. By evaluating complex network environments, such as those flooded by malicious IoT attacks (like those captured in the Kitsune Dataset [15]), it has been proven that unsupervised models can successfully identify anomalous behaviors by bypassing the need for labeled, perfectly balanced attack signatures. This mathematical approach treats anomalies purely as statistical outliers rather than requiring balanced training classes.

The current study leverages this inherent robustness against data sparsity by using Isolation Forest to handle the initial detection phase within the Ryu controller. This approach maintains high performance in live SDN environments where malicious signatures may represent less than 1% of the total flow volume. By focusing on the isolation of individual anomalous flows rather than classification against a known, balanced dataset, the cascaded system negates the requirement for computationally intensive real-time data resampling, yielding a lightweight and scalable defensive mechanism suitable for high-speed infrastructures.

Theoretical Background

This study is anchored in a multi-disciplinary theoretical framework that justifies autonomous mitigation of Distributed Denial-of-Service (DDoS) attacks. These are Control Theory, Extreme Value Theory (VET), Theory of Multiple Classifier Systems (MCS), and Information Theory. Together, these frameworks provide the conceptual basis for understanding the vulnerabilities of centralized networks and the mathematical justification for automated, real-time threat mitigation

The foundation of anomaly detection in this study is built upon **Information Theory**, as comprehensively formalized by T. M. Cover and J. A. Thomas [54]. Central to this mathematical theory is the concept of Entropy, which serves as a measure of uncertainty, randomness, or information content within a stochastic process or data

stream (as depicted in Figure 1). The theory dictates that any communication system possesses an inherent statistical structure; when subjected to external disturbances or "noise," the entropy of the data distribution shifts significantly.

Within the context of a Software-Defined Network (SDN), telemetry data extracted from OpenFlow switches such as packet rates, byte counts, and flow durations exhibits a predictable, low-entropy profile under normal operating conditions. However, a volumetric DDoS attack injects a massive influx of malicious traffic, creating a chaotic, high-entropy state. By monitoring these live telemetry streams for sudden increases in uncertainty or "information gain," the system's machine learning algorithms can mathematically trigger a classification event at the controller level.

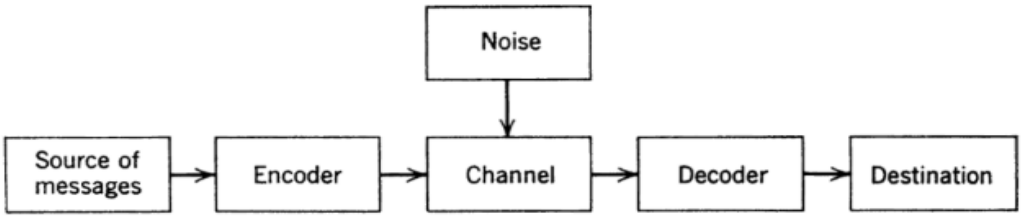

Figure 1. Information Theory Applying in Communication systems [54].

Once a high-entropy state is identified, the framework utilizes **Extreme Value Theory (EVT)**, specifically the Peak-Over-Threshold (POT) approach applied to anomaly detection by A. Siffer et al. [52]. Unlike traditional Gaussian statistics that focus on the "average" behavior of a system, EVT focuses exclusively on the "tails" of a distribution. The theory dictates that extreme statistical outliers (maxima or minima) can be mathematically identified and isolated in continuous data streams without making any prior assumptions about the "normal" distribution of the data.

These principles are applied through the Isolation Forest algorithm, an unsupervised machine learning model operating within the Software-Defined Network (SDN). During a Distributed Denial of Service (DDoS) attack, the sudden, massive influx

of malicious traffic creates extreme multivariable spikes such as abnormal packet rates, flow durations, and byte counts. Instead of comparing this traffic against pre-labeled attack signatures or assuming a predictable baseline of "normal" behavior, the system's machine learning model utilizes the mathematical principles of EVT to continuously evaluate the live telemetry extracted from the OpenFlow switches.This allows the system to isolate anomalous traffic in real time based on extreme statistical deviations.

This theory provides a mathematical basis for a dynamic boundary that adapts in real time, enabling the system to detect unpredictable, zero-day DDoS floods based on their extreme statistical characteristics. Once these anomalies exceed the defined threshold, a trigger mechanism forwards them to the classification stage for further verification.

To ensure that the extreme outliers detected via EVT are accurately categorized as malicious, this study is further anchored in the **Theory of Multiple Classifier Systems (MCS)**, established by J. Kittler et al. [53]. This machine learning theory proves that in complex, high-dimensional data environments, any single model is prone to specific misclassifications due to inherent algorithmic bias or "overfitting." The theory mathematically demonstrates that by aggregating the decisions of multiple independent classifiers through fusion rules as depicted in **Figure 2**, the combined system effectively neutralizes the isolated errors of each base model.

In this proposed framework, the triggered anomalies from the Isolation Forest for anomaly isolation are passed to the Random Forest for pattern classification for further verification. These variables are then fused to compute a final, highly stabilized classification. This provides the structural justification for how the system achieves superior True Positive Rates. By leveraging Kittler's decision-fusion principles, the controller can confidently distinguish between a legitimate "flash crowd" and a malicious DDoS flood, ensuring that defensive actions are only taken when a consensus of models confirms the threat.

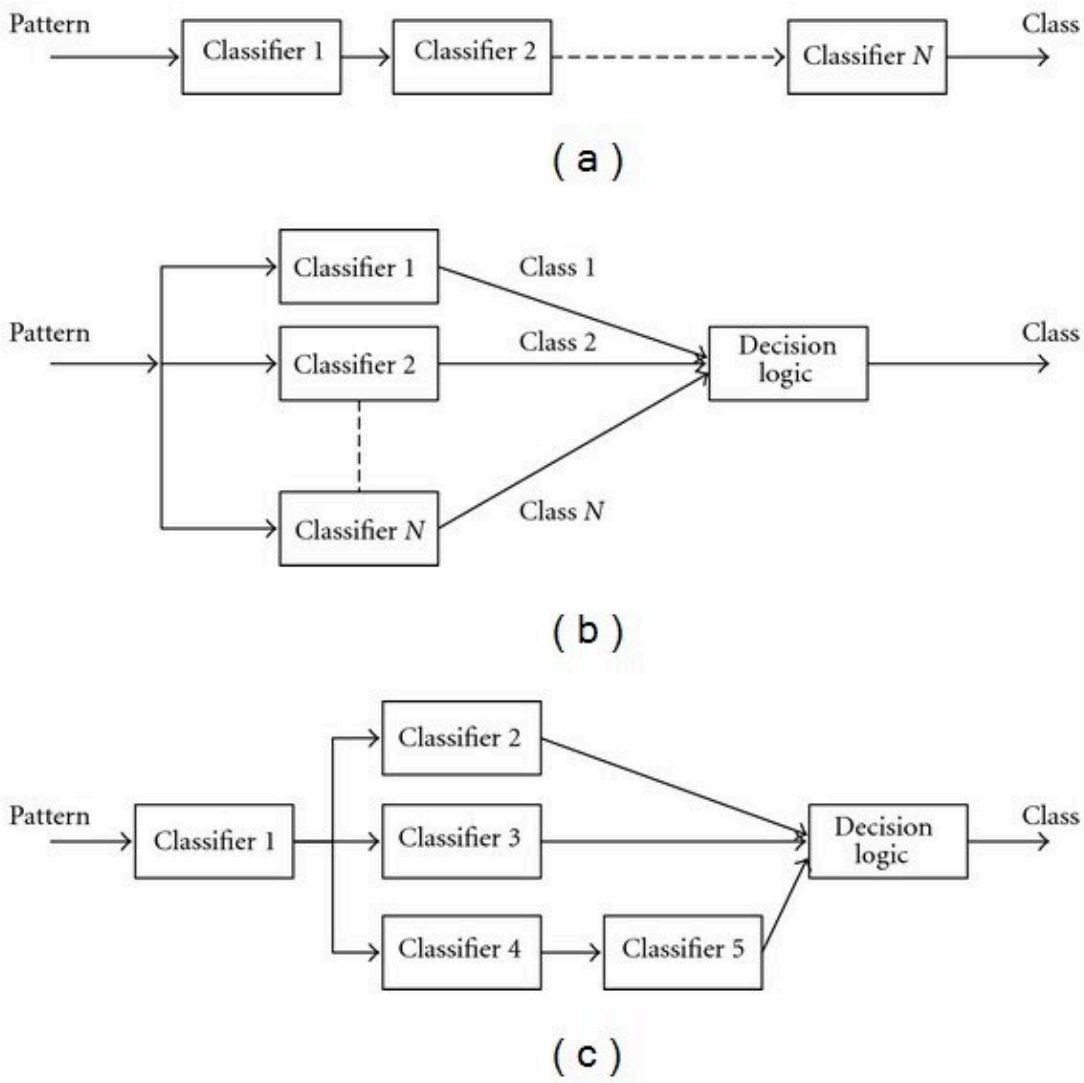

Figure 2.Theory of Multiple Classifier System [70].

The theoretical cycle concludes with **Control Theory** by Torkel Glad and Lennart Ljung [50], specifically the principles of multivariable and nonlinear control. This theory dictates how dynamic systems maintain stability by utilizing automated, closed-loop feedback mechanisms to counteract unpredictable disturbances as depicted in **Figure**

3. This is further supported by research from Moloja and Malele [51], who identify Control Theory as the optimal integrated framework for regulating AI-driven anomaly detection in next-generation networks.

Control Theory provides the engineering justification for the network's automated mitigation cycle. This corresponds to the plan and-execute phase of the control loop, where mitigation decisions are formulated and implemented. The SDN functions as a closed-loop multivariable control system: the Ryu controller serves as the central control unit, the OpenFlow switches function as the "plant," and the feature extraction module acts as the sensor. When a DDoS attack creates a nonlinear disturbance in variables like link utilization and flow table capacity, the controller acts as the actuator by deploying Flow Modification (FlowMod) rules. This corrective action, derived from control logic, restores the network to its optimal operating state. This ensures that the system doesn't just "detect" a threat, but "regulates" the network back to stability entirely without human intervention.

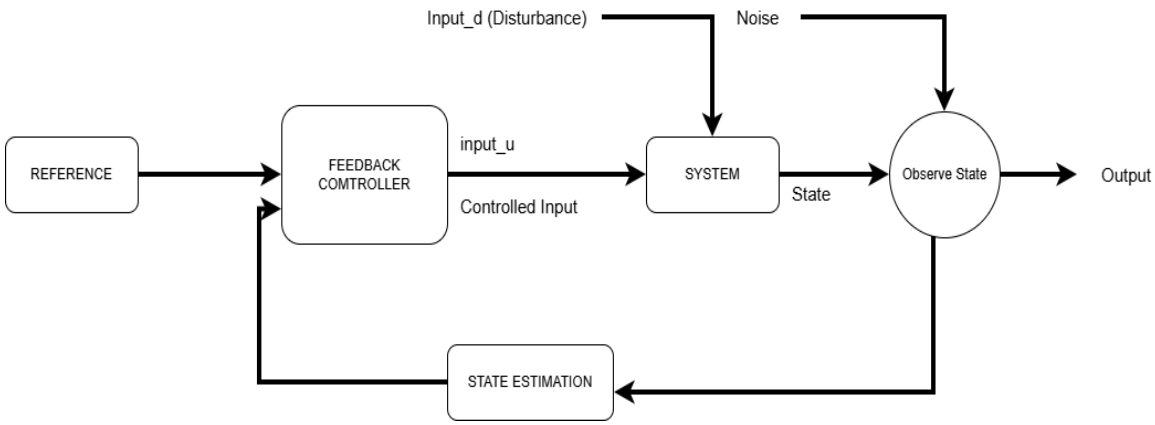

Figure 3. Control Theory [50]

To provide a strong and reliable foundation for autonomous network defense, the proposed system is based on a combination of established mathematical, statistical, and system-level theories.

The theories are integrated into a unified framework. As depicted in **Figure 4**, this framework shows the entire lifecycle of DDoS detection and mitigation. The process begins with Information Theory, which provides the foundation for monitoring network entropy and identifying deviations from normal behavior. Extreme Value Theory (EVT) is then applied to statistically define “tail events” or extreme outliers. To ensure high precision and reduce false positives, the Theory of Multiple Classifier Systems employs an ensemble of machine learning models to verify detected anomalies. Finally, the framework is completed through Control Theory, which manages the feedback loop between the Control Plane and the Data Plane, enabling the SDN controller to deploy precise FlowMod rules and restore network stability.

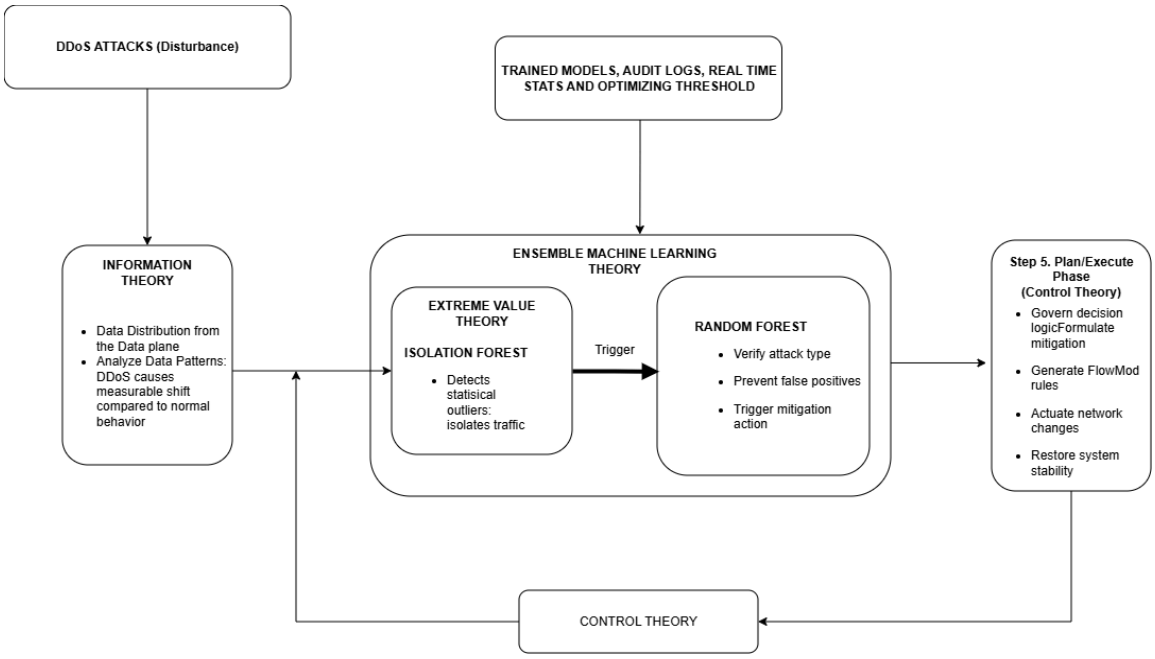

Figure 4. Theoretical Framework

Conceptual Framework

The systematic progression of the unsupervised-driven architecture for DDoS detection and mitigation will be delineated through the Input-Process-Output (IPO) model. This framework will illustrate the logical transitions from the initial resource gathering to the autonomous execution of defensive actions, ensuring a structured approach to network security.

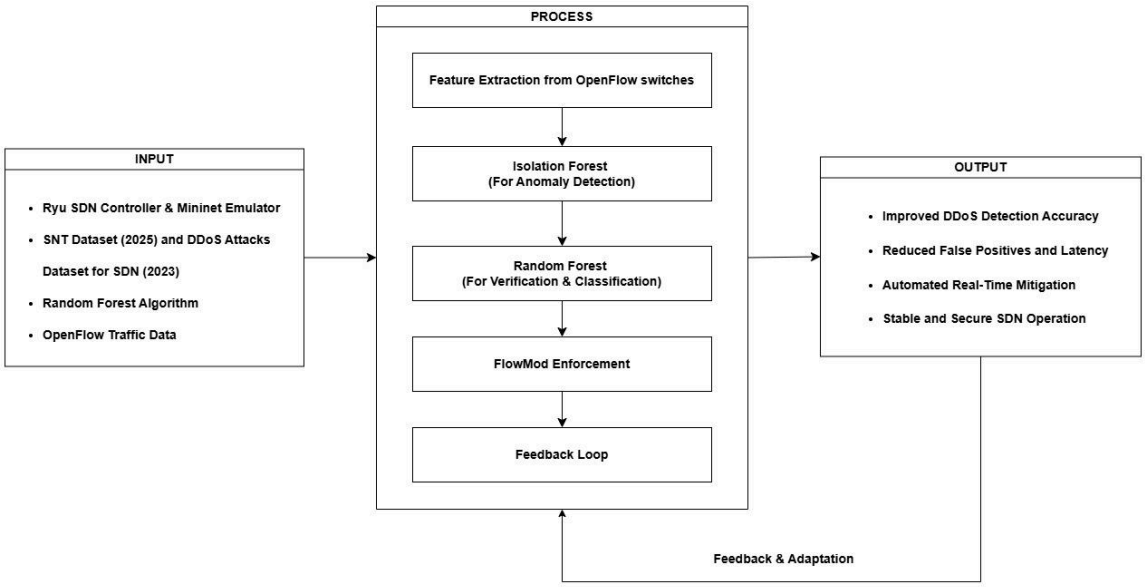

Figure 2. Input-Output-Process Model

As shown in **Figure 2,** the framework initiates with the **Input phase,** which encompasses the essential computational resources and data repositories required for system development. The software environment consists of the Ubuntu 22.04 LTS operating system, the Mininet network emulator, and the Ryu SDN Controller, which

together provide a programmable platform for network management. To facilitate baseline model training and validation, the system utilizes the Simulated Network Traffic (SNT) DDoS Detection Dataset (2025) and the DDoS Attacks Dataset (2023).

Furthermore, during active simulation, real-time OpenFlow traffic statistics including packet counts, byte rates, and flow durations are continuously ingested from the virtual switches to serve as the primary live telemetry for the analytical engine.

The **Process phase** represents the core operational logic where raw network telemetry is transformed into definitive defensive actions. This stage begins with Feature Extraction, where the polled switch statistics are converted into structured formats compatible with the machine learning algorithms. A cascaded, two-tier detection mechanism is then executed: the unsupervised Isolation Forest algorithm first identifies statistical anomalies, which are subsequently relayed to the dormant Random Forest classifier for rigorous threat verification. Upon the confirmation of a malicious signature, the Ryu Controller triggers the Autonomous Mitigation module by pushing instantaneous Flow Modification (FlowMod) directives to the OpenFlow switches, thereby dropping malicious packets and neutralizing the threat in real-time.

Finally, the **Output phase** manifests the quantifiable results and the overall security posture of the hybrid framework. The measured outcomes include enhanced detection accuracy, a reduction in false-positive rates, mitigation latency speeds, and the sustained throughput of legitimate traffic during active attack scenarios. Central to this architecture is a closed-loop feedback mechanism, represented in **Figure 2** by a logical path returning from the Output back to the Process phase. This loop allows the system to evaluate its own performance metrics and dynamically recalibrate its detection thresholds. Consequently, the framework achieves a state of continuous adaptation, ensuring that the SDN environment remains resilient against volumetric DDoS floods without the necessity of manual administrative intervention.

Synthesis

The reviewed literature indicates that network security is undergoing a significant transformation from rigid, hardware-dependent defenses toward adaptive, software-defined intelligence. A common finding across the literature is that the centralized Software-Defined Networking (SDN) controller serves as a critical single point of failure. This architectural characteristic introduces a significant vulnerability, where a volumetric Distributed Denial-of-Service (DDoS) attack can disrupt or even collapse the entire network infrastructure. As a result, there is a strong consensus that traditional signature-based detection methods are no longer sufficient, particularly against evolving or zero-day threats. Consequently, current research increasingly emphasizes the adoption of data-driven, anomaly-based machine learning techniques.

Within this transition, studies consistently highlight a trade-off between computational efficiency and detection accuracy. Unsupervised models are effective in rapidly identifying novel anomalies without requiring labeled datasets, whereas supervised models achieve high classification accuracy but often impose significant computational overhead, especially when processing large-scale and imbalanced traffic data. Furthermore, the literature emphasizes that deploying resource-intensive detection mechanisms can unintentionally degrade network performance by overloading the SDN controller, thereby creating a self-inflicted bottleneck during high-volume attacks.

Despite these advancements, a critical research gap remains in integrating detection and mitigation into a unified, closed-loop framework. Existing studies often treat anomaly detection and response as separate processes, leading to delays between identification and action. Few approaches effectively combine the speed of unsupervised detection with the accuracy of supervised classification while enabling

real-time automated mitigation. Addressing this gap is essential to achieve both high detection accuracy and rapid response, ensuring network resilience in SDN environments.

Definition of Terms

The following concepts are operationally and conceptually defined for the study:

Conceptual Terms

Terms	Definition
Anomaly-Based Detection	An analytical technique used to identify data points or behaviors that deviate from expected patterns, focusing on discovering irregularities that may indicate underlying problems or abnormal system behavior [55].
Benign Traffic	Refers to legitimate, safe, and standard data packets generated by authorized users or applications. It serves as the mathematical baseline of "normal" operational behavior in Anomaly-Based Detection systems, which must differentiate these safe flows from malicious outliers to prevent false positive alerts [8], [10]
Closed–Loop	A control system architecture that utilizes a continuous, automated feedback mechanism to regulate its operational state without human intervention. [50]
Distributed Denial of Service (DDoS)	A type of attack where multiple systems target a single system to flood the targeted resource with traffic and cause a denial of service [57].

Ensemble Machine Learning	A computational paradigm where multiple, distinct machine learning algorithms (base learners) are strategically cascaded or combined to solve a single predictive task [52].
Flow Modification (FlowMod)	A mechanism within software-defined networking where a controller sends messages to a switch to dynamically add, update, or delete flow entries in its flow table to manage traffic [58].
Hybrid Machine Learning	A machine learning approach, such as semi-supervised learning, that integrates different types of models combining supervised and unsupervised techniques to improve predictive accuracy [59].
Isolation Forest	A useful and efficient unsupervised algorithm used for anomaly detection that works by isolating data points that differ significantly from normal observations using random partitioning [60].
Mininet	A network emulator used to create virtual network topologies that allows the simulation of complex networks on a single machine by creating virtual hosts, switches, routers, and links [62].
OpenFlow	A communications protocol that allows for the remote access and management of a network switch or router's forwarding plane by a separate controller [63].
Random Forest	An ensemble machine learning algorithm that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting [61].

Software-Defined Networks (SDN)	An approach to networking that uses software-based controllers or APIs to communicate with underlying hardware infrastructure and direct traffic, decoupling the control plane from the data plane [58].
Unsupervised Machine Learning	A type of machine learning where the model works without labeled data, learning patterns on its own by grouping similar data points or finding hidden structures without human intervention [64].
Zero-Day Attack	A type of security threat that exploits a software vulnerability that is not yet known to the vendor or the public, occurring before a patch or defense can be created [65].

Operational Terms

Terms	Definition
Application Plane	This refers to the topmost logical layer in your framework where the Python-based security scripts, specifically the cascaded Isolation Forest and Random Forest machine learning models reside and execute.
Control Plane	It acts as the intermediary "governor" that receives the security decisions from the Application Plane (the ML models) and translates them into technical OpenFlow (FlowMod) commands to physically program the switches.

Data Plane	Refers to the layer of Mininet virtual switches responsible for carrying all underlying network traffic. Its sole operational purpose is to forward or drop these packets strictly based on the FlowMod rules commanded by the central Ryu controller
False Negative Rate (FNR)	This is the percentage of actual malicious DDoS flows that the detection engine fails to identify. A high FNR means attack traffic is successfully bypassing the Ryu controller's defenses and saturating the network.
False Positive Rate (FPR)	Evaluate how often the cascaded machine learning system incorrectly classifies legitimate background traffic as a DDoS attack, which would result in the unjustified dropping of benign packets.
Feature Extraction	The operational process where the Ryu controller continuously polls the Mininet switches to gather specific raw network metrics and formats them into structured data arrays for the algorithms to analyze.
Mitigation Response Time	Defined as the measurable time delay from the exact moment the machine learning engine flags a threat to the moment the Ryu controller successfully installs the blocking FlowMod rule in the Mininet switch.
Ryu Controller	This is the specific Python-based SDN controller software deployed in the Ubuntu virtual machine to host the machine learning models, analyze incoming OpenFlow traffic statistics, and orchestrate network defenses.
Single Point of Failure (SPOF)	This concept represents the inherent architectural vulnerability of the Ryu controller, which the

	experiment aims to protect from CPU exhaustion and crashing during high-speed traffic floods.
True Positive Rate (TPR)	Defined as the proportion of actual, simulated DDoS attack flows (such as TCP-SYN or UDP floods) that the machine learning models correctly identify, flag, and successfully mitigate during the active Mininet simulation.
Volumetric Attack	Designed to overwhelm the virtual network's bandwidth and test the processing limits of the controller.

CHAPTER III

RESEARCH METHODOLOGY

This chapter outlines the research design and systematic procedures utilized to develop and evaluate the cascaded machine learning framework for DDoS mitigation within a Software-Defined Networking (SDN) environment. Utilizing an Exploratory Sequential Mixed Methods framework, the study converts qualitative industry demands into a regulated quantitative experiment within a virtual sandbox environment. Development adheres to an iterative technique, advancing from the preliminary network topology design in Mininet to the successful prototype integration of the Isolation Forest and Random Forest algorithms. This section outlines the purposive sampling of IT professionals for the ISO 25010 software quality assessment, the technical evaluation of the "Decision-to-Action" protocol, and the statistical methods employed to validate both the algorithmic precision and the system's practical feasibility.

Research Design

The research will employ an Exploratory Sequential Mixed Methods design to systematically develop and assess the proposed unsupervised machine learning framework for Distributed Denial-of-Service (DDoS) mitigation. This approach is highly appropriate for this study, as it initiates with a qualitative exploration to establish operational requirements, which directly informs the subsequent quantitative development and evaluation phases.

The study will initiate with a qualitative phase involving semi-structured interviews with network and IT administrators located in Laguna. The primary objective of this initial phase is to gather baseline data regarding current industrial challenges, such as alert fatigue from false positives and the operational delays inherent in manual threat responses. This qualitative data is crucial, as it establishes the practical parameters and enterprise requirements that the automated mitigation system must address.

The research will then transition into the quantitative and experimental phase. The cascaded machine learning framework will be developed and deployed within an isolated virtual sandbox environment utilizing Ubuntu 22.04 LTS, the Mininet emulator, and the Ryu SDN Controller. During this phase, the system will be subjected to simulated volumetric DDoS attacks using standard datasets. Quantitative data, including detection accuracy, false-positive rates, mitigation latency, and controller CPU overhead, will be rigorously measured to objectively evaluate the algorithm's technical efficacy.

To complete the sequence, the fully developed system and its web-based visualization dashboard will be presented back to the panel of Network Admins. The system's practical viability and alignment with the initial industry requirements will be quantitatively measured using an evaluation survey adapted from the ISO/IEC 25010 software quality model. This final assessment ensures that the engineered solution effectively resolves the qualitative problems identified at the beginning of the study.

Research Locale

The primary physical locale for the conduct of this research is the Province of Laguna, where the researchers will conduct the development and validation phases of the study using a locally hosted virtualized environment. Due to the nature of network security experimentation, which requires a safe and isolated testing environment, the

system's demonstration and technical simulation will be performed within a controlled virtual sandbox.

The Software-Defined Network (SDN) will be implemented locally on a host machine running Ubuntu 22.04 LTS. Within this environment, the Mininet network emulator and the Ryu SDN Controller will be utilized to orchestrate the network topology, demonstrate system functionality, and execute simulated Distributed Denial-of-Service (DDoS) attacks without affecting real-world network infrastructures

Respondents of the Study

The respondents of this study will consist of a purposely selected group of twelve IT professionals, divided into two distinct categories: Network Administrators and IT Experts. This classification is designed to ensure a balanced evaluation of the proposed system from both operational and technical perspectives.

Classification	**Number of Participants**
Users	10
IT Experts	2
Total	12

Table 1. Respondents of the Study

The first group is the users composed of ten (10) Network Administrators or IT Administrators currently employed in enterprise, corporate, or institutional environments. These respondents are selected based on their direct involvement in network operations, monitoring, and incident response. To qualify, participants must have at least three (3) to five (5) years of professional experience in managing network infrastructure,

handling traffic analysis, and responding to cybersecurity threats such as Distributed Denial-of-Service (DDoS) attacks. Additionally, they must possess working knowledge of Software-Defined Networking (SDN), particularly in relation to centralized control plane architectures. This group is tasked with evaluating the system's usability, operational efficiency, response time, and overall practicality in real-world deployment scenarios.

The second group consists of evaluators of the system consisting of two (2) IT Experts with demonstrated specialization in at least one of the following domains: Cyber Security, machine learning, or Software-Defined Networking (SDN). These experts must have more than five (5) years of professional experience in their respective fields, ensuring a high level of technical proficiency and domain mastery. There are also factors selected based on advanced qualifications, which may include postgraduate education, professional certifications (e.g., CCNA, CEH, or equivalent), system development experience, or research involvement in relevant areas. Unlike the administrators, their role is to critically evaluate the system's technical soundness, including the correctness of the cascaded machine learning approach, the integration of Isolation Forest and Random Forest algorithms, and the effectiveness of the automated mitigation mechanism within the SDN architecture.

Data Gathering Procedures

Instrumentation

The data collection process for this research employs a multi-methodological approach, integrating high-fidelity simulation software with a qualitative assessment framework. This strategy ensures a holistic evaluation by combining empirical performance data with expert technical perspectives.

The primary tool for collecting technical data is a virtualized Software-Defined Networking (SDN) testbed. This environment is constructed using the Mininet network emulator and the Ryu SDN controller running on an Ubuntu Linux platform. To simulate realistic network stress and evaluate the cascaded detection architecture, the study utilizes two specialized datasets. First, the "DDoS Attacks Dataset for Software Defined Network" is employed to provide standardized attack signatures tailored for SDN environments. Second, the "SNT (Simulated Network Traffic Using Mininet and Ryu) DDoS Detection Dataset" is used to provide high-fidelity traffic patterns specific to the chosen controller and emulator environment.

The data collection procedure is automated through a custom-built monitoring module within the Ryu controller. This instrument programmatically captures flow statistics such as packet arrival rates, byte counts, and flow durations via the OpenFlow protocol. These metrics provide the raw quantitative data required to evaluate the detection precision and resource efficiency of the Isolation Forest and Random Forest algorithms.

To gather structured feedback on the system's software quality, a survey instrument adapted from the ISO 25010 standard is employed. This tool is designed to measure three specific quality characteristics: Functional Suitability, Performance Efficiency, and Security. The instrument consists of a series of indicator statements where respondents consisting of IT professionals and network experts evaluate the system using a four-point Likert scale.

This forced-choice format is specifically chosen to eliminate neutral responses, ensuring that the gathered data provides a clear consensus on the system's operational readiness and its success in preventing controller resource exhaustion.

To complement the quantitative findings, the study utilizes semi-structured interviews to gather technical insights from network administrators and IT experts. This process is guided by a predetermined set of open-ended questions maintained internally

by the researchers to ensure consistent coverage of core domains, such as the "Decision-to-Action" protocol and system reliability during volumetric floods.

Unlike a standardized survey, this instrument serves as a flexible framework for the interviewer rather than a direct questionnaire for the respondent. This approach facilitates spontaneous follow-up inquiries into operational nuances that numerical data cannot capture, including the practical challenges of deploying a cascaded machine learning architecture in a production environment. By balancing thematic consistency with flexible dialogue, the study ensures a granular assessment of the system's real-world viability.

Validation

The researchers will use datasets of simulated network traffic for systematic processing and evaluation. To evaluate the system's efficacy, simulated network traffic datasets will be processed using the Ryu SDN controller and Mininet emulator, integrated with a cascaded machine learning architecture. During emulation, the unsupervised Isolation Forest model will first identify statistical anomalies in OpenFlow traffic to filter suspected volumetric floods. Flagged anomalies will then be iteratively processed by the supervised Random Forest algorithm, mapping them against historical signatures to confirm specific Distributed Denial-of-Service (DDoS) threats and trigger automated mitigation.

The technical validity of the system will be established by computing its detection accuracy, false-positive rates, and controller CPU utilization during these processing cycles.

The survey instrument utilized for the expert evaluation holds strong inherent validity, as it is strictly adapted from the internationally recognized ISO/IEC 25010 Systems and Software Quality Requirements and Evaluation (SQuaRE) model. To

ensure that the specific indicator statements and semi-structured interview questions are clear, unbiased, and perfectly aligned with the study's objectives, they will undergo rigorous content validation. Prior to distribution to the target population of network administrators and IT professionals, the initial drafts will be submitted to the thesis adviser and a select panel of academic validators. These experts will review the phrasing, technical terminology, and the appropriateness of the four-point Likert scale. Any ambiguities or misalignments identified during this review will be revised and refined, ensuring the final instrument accurately captures the respondents' assessment of the cascaded machine learning framework's Functional Suitability, Performance Efficiency, and Security.

Discussions on Algorithms/Mathematical Concepts Used/Proposed Solutions

Fundamentals of the Algorithms/Mathematical Model/Formula

The proposed system uses a dual approach to secure software-defined networks (SDNs) against distributed denial-of-service (DDoS) attacks. The system combines the Isolation Forest algorithm for unsupervised anomaly detection, and if an anomaly is detected, the Random Forest is for classification of what type of attack the anomaly is.

The primary algorithm used is Isolation Forest. We train Isolation Forest specifically on benign or normal traffic. By learning the patterns of normal networks, the algorithm learns what healthy traffic looks like. When a DDoS attack occurs, the malicious flows are isolated much faster than normal flows because they do not fit the normal traffic used to train. Once Isolation Forest flags the traffic as an anomaly, the random forest is triggered. The Random Forest acts as a classifier, using its decision trees to determine the category of the attack, such as a SYN flood or UDP flood.

The proposed system solves the problem of detecting zero-day attacks. Traditional systems usually look for specific signatures of an attack. If the attack signature is not known yet, the traditional system might fail to detect the attack. The method used by the proposed system solves these problems because it does not need to know what an attack looks like. It only knows what normal traffic looks like. Anything else is flagged as an anomalous threat to SDN.

Additionally, this feature addresses the issue of operational cost. Because the Isolation Forest is a lightweight algorithm, the Random Forest will only activate after detecting attacks, reducing the computational power needed for detection. This feature can lessen the operational cost while providing the high-level protection of the SDN controller.

The proposed system extracts features directly from the SDN controller's flow tables to analyze network behavior. The extracted features fall into five categories:

- **Time and Duration**: This tracks how long a flow stays active in the switch memory. Features like flow_duration_sec, flow_duration_total_ns, idle_timeout, and hard_timeout define the exact lifespan and timeout rules of the connection.

- **Volume and Counts:** This defines the sheer bulk of the data moving through the network. We measure the raw size of the connection using packet_count, byte_count, and disp_pakt.

- **Speed and Rates:** DDoS attackers usually send data much faster than normal users. To catch this sudden rhythm, monitor velocity features like packet_count_per_second, byte_count_per_second, and interval rates like pkt_per_interval and byte_per_interval.

- **Packet Characteristics and Ratios:** This category measures the physical shape and proportionality of the data frames. By looking at bytes_per_packet, mean_byte, and metrics like pkt_byte_rate_ratio and pkt_size_bucket, we can easily spot the unnatural, repetitive packet sizes that are very common in flood attacks.

- **Network Diversity:** This monitors how the traffic interacts with the control plane and checks for spoofing behaviors. Features here include the flow_entry_ratio along with OpenFlow flags to observe how the switch resources are being stressed.

The proposed system processes these variables as inputs to produce a final label: 0 for normal or a specific attack category (e.g., SYN flood or UDP flood).

To see this in action, picture a sudden increase in network traffic. Most of this traffic consists of regular users accessing a website, but hidden among them is a "UDP Flood" attack trying to crash the server. The Isolation Forest monitors the flows and notices that the attack packets move with a repetitive, mechanical speed that looks nothing like the benign data it learned during training. It immediately isolates these flows and raises an alarm. The Random Forest then steps in to analyze the 18 features of the suspicious traffic. It recognizes the specific UDP protocol patterns and identifies the threat as a "UDP Flood," which is a type of attack that overwhelms a target with a large number of UDP packets. Within seconds, the SDN controller receives this classification and blocks the malicious source, allowing regular users to continue browsing without any lag.

By using Isolation Forest as an anomaly detector, the system avoids wasting energy on constant, heavy analysis.It only uses the more complex random forest when something truly suspicious appears to classify the anomaly. This design keeps the SDN controller healthy during traffic spikes.

Theoretical Foundations

The proposed system is scientifically grounded in two established machine learning theories, anomaly detection and ensemble learning. These theories directly support the design choices made in the proposed cascaded architecture and explain

why Isolation Forest and Random Forest were selected as the primary algorithms for detection and classification respectively.

Anomaly detection is the theoretical basis for the first phase of the system. It operates on the principle that normal network behavior follows a consistent and recognizable pattern, while anomalous traffic is statistically rare and distinct enough to stand apart from that pattern [11]. A model trained only on normal traffic can therefore flag any flow that deviates significantly from what it has learned as a potential threat, without ever needing labeled attack examples. Isolation Forest implements this principle through a mechanism called isolation, where each isolation tree is built by randomly selecting a feature and a random split value to recursively partition the data [38]. Because anomalous data points are few and statistically different, they tend to get separated from the rest of the data in very few splits, resulting in shorter average path lengths across the isolation trees. Normal data points require more splits to isolate and therefore produce longer path lengths. The anomaly score of each flow is computed from this average path length, with shorter paths corresponding to a higher likelihood of being an anomaly [38]. This design gives Isolation Forest a linear time complexity and a small memory footprint, making it well suited for deployment inside a resource-shared SDN controller environment [10], [15].

Ensemble learning is the theoretical basis for the second phase of the system. It is grounded in the principle that combining the predictions of many individual models produces a stronger and more reliable result than any single model alone [40]. Random Forest implements this principle by training an ensemble of decision trees, where each tree is built on a different bootstrapped sample of the training data and evaluates only a randomly selected subset of features at each split node [40].

This combination of bagging and random feature selection reduces overfitting, improves generalization across unseen data, and makes the model robust to class imbalance in the training data [39]. These properties make Random Forest particularly

well suited for multi-class attack classification, where attack classes may vary significantly in their sample sizes and feature distributions.

Anomaly detection addresses the limitation of supervised learning by identifying unknown threats without labeled data, while ensemble learning addresses the limitation of unsupervised detection by providing high-precision classification to reduce false positives. The combination of both theories results in a system that is both flexible and reliable, capable of detecting novel attacks while minimizing unnecessary disruption to legitimate network traffic.

Algorithm Design

This study will establish a structured, multi-step sequential machine learning algorithm to solve the challenge of Distributed Denial of Service (DDoS) attacks without overloading the central network controller. To ensure a balance between processing limits and operational detection efficacy, the workflow will function in a continuous cycle, assessing data through critical automated decision points. The algorithm will follow an organized progression divided into four distinct phases: initial system preparation, continuous data iteration, anomaly detection, secondary threat verification, and automated rule-based enforcement

Phase 0: Algorithm and Dataset Preparation

The foundation of the detection mechanism relies heavily on the environment setup and data initialization. Because the system utilizes static models to stay within processing bounds, the system operations must be prepared prior to network deployment.

Step 1: Environment and Baseline Initialization. The system initiates the software environment by configuring the Mininet Emulator and deploying the Ryu SDN Controller, establishing a simulated testing ground. Physical hardware

deployment and enterprise-level testing outside the simulated environment are explicitly excluded. Following this, the machine learning algorithms, specifically the Isolation Forest and Random Forest models are subjected to a static offline training phase using the pre-established dataset to build the mathematical boundaries for normal and anomalous traffic. No dynamic updates or continuous real-time learning algorithms are utilized in the active environment.

Phase 1: Continuous Data Iteration

Once initialized, the workflow shifts into an active monitoring state, continuously processing incoming data from the simulated network.

Step 2: Network Monitoring and Extraction. The centralized Ryu controller initiates a continuous polling loop, systematically collecting real-time traffic statistics from all connected OpenFlow network switches. To preserve host machine resources and avoid overloading the controller, the system bypasses deep packet inspection and any form of application-layer payload examination. The controller specifically extracts Network and Transport Layer metadata explicitly packet counts, byte rates, and flow durations.

Step 3: Data Formatting. These gathered metrics are then formatted into structured statistical arrays required by the subsequent machine learning models.

Phase 2: Primary Detection via Isolation Forest

The formatted statistical arrays are passed to the primary detection layer for initial screening.

Step 4: Anomaly Evaluation. The unsupervised Isolation Forest model receives the structured data arrays from the controller. The algorithm evaluates the structural attributes of the incoming flows, analyzing how significantly the current patterns deviate from the established static baselines without actively querying external threat lists.

Step 5: Score Generation. Based on this evaluation, the Isolation Forest generates an anomaly score s for the specific data stream[38]. This is calculated using the following formula:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

Phase 3: Decision Gateway and Classification layer

The algorithm reaches a critical decision gateway, where actions are governed directly by the anomaly score generated in Phase 2.

Step 6: Anomaly Interpretation. The system interprets the anomaly score. If the traffic evaluates as normal (where the score s < 0.5), the system takes no further action on that data stream and initiates a closed feedback loop back to Phase 1, conserving processing power.

Step 7: Threat Validation. If the traffic is evaluated as anomalous (where the score s is close to 1), the system flags the flow and triggers the classification layer process to mitigate false positives. The flagged traffic data is forwarded to the supervised Random Forest algorithm.

Step 8: Final Classification. The Random Forest analyzes the anomalous features and cross-references them against its static historical training data. The

model definitively categorizes the flow as either a legitimate burst or a verified DDoS flood by functioning on the principle of majority vote across its decision trees[40]. This verification operates within the confines of its initial setup parameters and is formulated as:

$$\hat{y} = mode\,\{h_1(x),\ h_2(x).,\ldots,\ h_B(x)\}$$

Phase 4: Automated Rule-Based Mitigation

Upon the Random Forest successfully validating a DDoS threat, the system initiates the final enforcement phase to protect the network.

Step 9: Enforcement Execution. The Ryu controller generates a targeted Flow Modification (FlowMod) rule specifically designed to block the identified malicious source. Operating strictly within the constraints of the simulated emulator environment, the controller autonomously dispatches this rule down to the specific virtual OpenFlow switches handling the attack traffic. The virtual switches immediately execute the instruction, dropping the malicious packets. This blocks the Network and Transport layer attacks before the traffic can cause host controller saturation, completing the automated defense loop.

Pseudocode:

The comprehensive step-by-step logic is represented in the following pseudocode:

INPUT: OpenFlow Traffic Statistics (packet_count, byte_rate, flow_duration)

OUTPUT: FlowMod Enforcements (block/allow) + Evaluation Log

PHASE 0: ALGORITHM AND DATASET PREPARATION

STEP 1: Initialize System and Offline Baselines

INITIALIZE Mininet Emulator AND Ryu SDN Controller

```
dataset_training_array = LOAD(dataset)

TRAIN(Isolation_Forest, dataset_training_array)

TRAIN(Random_Forest, dataset_training_array)
```

PHASE 1: CONTINUOUS NETWORK MONITORING AND EXTRACTION

STEP 2: Collect Traffic Statistics

LOOP CONTINUOUSLY:

```
traffic_data = POLL(OpenFlow_Switches)
```

STEP 3: Extract and Format Metadata

```
metadata = EXTRACT(traffic_data.packet_counts,
traffic_data.byte_rates, traffic_data.flow_durations)

structured_array = FORMAT_FEATURES(L3_L4_metadata)
```

PHASE 2: PRIMARY DETECTION VIA ISOLATION FOREST

STEP 4: Evaluate Structured Array

```
EVALUATE structured_array IN Isolation_Forest AGAINST
static_baselines
```

STEP 5: Generate Anomaly Score

```
s = CALCULATE_ISOLATION_SCORE(structured_array)
```

PHASE 3: DECISION GATEWAY AND SECONDARY VERIFICATION

STEP 6: Gateway Interpretation

```
IF s < 0.5 THEN
    action = NORMAL
    CONTINUE LOOP // Conserve processing power
END IF

IF s >= THRESHOLD_APPROACHING_1 THEN
    flag_status = ANOMALOUS
    FORWARD structured_array TO Random_Forest
END IF
```

STEP 7: Threat Validation

```
classification = RF_MAJORITY_VOTE(structured_array)
flow_status = CATEGORIZE(classification)
```

PHASE 4: AUTOMATED RULE-BASED MITIGATION

STEP 8: Execute Mitigation Strategy

```
IF flow_status == DDoS_Flood THEN

      flowmod_rule = GENERATE_FLOWMOD(malicious_source)

      DISPATCH flowmod_rule TO virtual_OpenFlow_switches

      EXECUTE  DROP_PACKETS  (virtual_OpenFlow_switches)  //
      Protects controller

END IF
```

The primary inputs are passive network flow characteristics such as packet counts, byte rates, and flow durations. The final output is an automated network rule that instructs the switches to block and drop the malicious packets.

Inputs: The algorithm takes raw network flow statistics gathered directly from the network switches. Specifically, these inputs are packet counts, byte rates, and flow durations.

Intermediate Steps: The first intermediate step is feature extraction, which formats the raw statistics into an array. The second is the anomaly score generated by the frontline Isolation Forest algorithm. If an anomaly is flagged, the final intermediate step is the threat classification output generated by the Random Forest algorithm.

Outputs: The final output is an automated Flow Modification command. This is an instruction sent to the network switches to drop the identified malicious packets and block the attack.

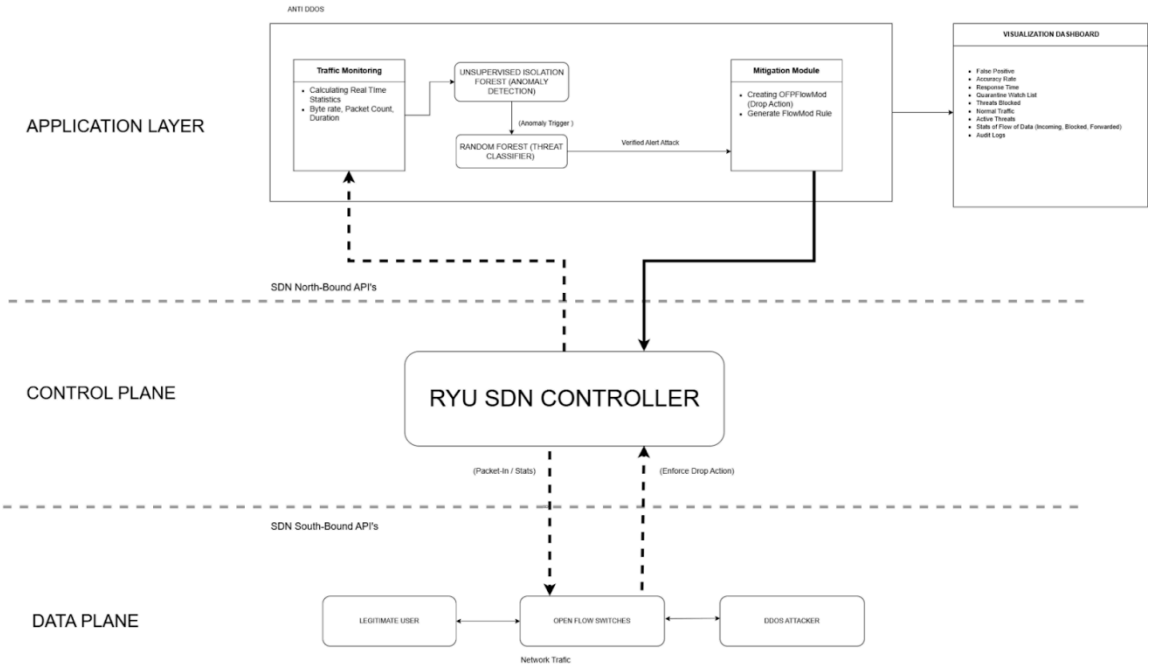

Figure 2. Block diagram illustrating the workflow of Isolation Forest and Random Forest DDoS mitigation algorithm

This flow design was chosen specifically to address the processing limits of a centralized network controller. Running heavy classification models continuously can cause the controller to crash during a massive traffc flood.

By using the lightweight Isolation Forest as a continuous monitor, the system saves valuable computer resources. The heavier Random Forest is only activated when a potential threat is found. This specific structure ensures high detection accuracy and minimizes false alarms while keeping the network running smoothly for legitimate users.

Mathematical Model Formulation

This section will formalize the mathematical models that will govern the proposed anomaly-based DDoS mitigation system. To ensure precision and logic, the system will utilize a sequential two-layer mathematical framework. The primary layer will calculate an anomaly score to filter out normal traffic, while the secondary layer will use an ensemble voting mechanism to finalize the threat classification and trigger mitigation.

Primary Detection Layer: Isolation Forest Anomaly Scoring

The frontline of the algorithm will utilize the Isolation Forest model. Instead of mathematically defining what normal network traffic looks like, this model will actively isolate anomalies based on the principle that malicious DDoS flows exhibit distinct, highly unusual structural patterns.

When the algorithm processes network traffic into decision trees, these unusual attack patterns will require fewer structural splits to be separated from the rest of the data, resulting in a significantly shorter traversal path [38].

To quantify this, the model will calculate an anomaly score, S, for every incoming network flow. The mathematical formula, as established by Liu, Ting, and Zhou [38], that will describe this scoring process is:

$$s(x,\ n)\ =\ 2^{-\frac{E(h(x))}{c(n)}}$$

where:

- $s(x,\ n)$: Represents the calculated anomaly score for a specific traffic flow.
- x : The specific traffic observation being checked.
- $h(x)$: The path length required to isolate the observation x within a decision tree.

- $E(h(x))$:The average number of steps it takes to isolate x across all the trees in the forest.

- $c(n)$: The average path length of an unsuccessful search in a Binary Search Tree given n external nodes. Which acts as a standard value to help scale the final score.

- n : A mathematical normalization constant representing the average path length of an unsuccessful search in a standard Binary Search Tree given nodes. This constant will scale the final score to ensure consistency.

The calculated anomaly score will be strictly constrained to fall within a mathematical boundary of $0 \leq s \leq 1$ [38]. The system will operate on the following predefined thresholds to conserve the Ryu controller's processing power:

- **Normal Boundary** ($s < 0.5$): If the score is low, the traffic will be evaluated as normal. The algorithm will terminate further checks on this flow and loop back to monitor new traffic.

- **Anomaly Boundary** ($s \approx 1$): If the score approaches 1, the traffic will be flagged as structurally anomalous and will be forwarded to the secondary validation layer.

Secondary Validation Layer: Random Forest Classification

If the primary layer flags a flow as anomalous, the system will forward the data to the supervised Random Forest algorithm [40]. The objective of this secondary model will be to classify the anomaly definitively, thereby minimizing false positives before any network rules are applied.

This model will rely on a discrete mathematical voting mechanism across an ensemble of independent decision trees [39]. Each tree will analyze the anomalous features against historical training data and cast a vote. The final definitive prediction, ŷ, will be determined by the most frequent vote, formulated as [40]:

$$\hat{y} = mode\,\{h_1(x),\ h_2(x).,\ldots,\ h_B(x)\}$$

where:

- $\hat{y}$: The final definitive categorization (legitimate burst or DDoS flood).
- x : The flagged anomalous traffic array passed from the primary detection layer.
- B : The total number of decision trees in the Random Forest.
- $h_B(x)$: The discrete classification output which is the specific vote cast by the B -th individual decision tree regarding observation x.
- $mode\,\{\ldots\}$: A standard statistical function that selects the most frequent vote among all the trees in the forest.

Since this system's objective is to automate network defense, the formula's output will be strictly constrained to a binary categorical decision. It must classify the flow as either a legitimate burst or a verified DDoS flood. If the mathematical mode evaluates to a verified DDoS flood, this boundary condition will serve as the absolute trigger to initiate the automated mitigation phase, generating the OpenFlow drop rule.

Algorithm Implementation

Programming Language

The detection and mitigation system will be developed using Python as the main programming language. Python was chosen for its comprehensive support for machine learning development, powerful data processing capabilities, and native compatibility for easy integration with the Ryu Software-Defined Networking (SDN) controller in resource-constrained environments.

Development Environment

Visual Studio Code (VS Code) will serve as the development environment for the whole development process. Version control will be handled by Git, with the project repository posted on GitHub to monitor algorithm iterations. To create the real network architecture, the system will use the Mininet network emulator, which runs on an Ubuntu virtual machine. Using Mininet and Ryu, the research assures that all packet processing and rule enforcement are performed natively, eliminating the usage of any third-party firewall programs.

Frameworks and Libraries

These frameworks and libraries were carefully chosen because they can quickly process massive amounts of data, run complex mathematical formulas directly inside the simulated network, and allow administrators to monitor the system locally. By using these built-in python libraries, the system successfully avoids the need for using third party software. The specific libraries that will be utilized and their distinct roles within the system are detailed in the table below:

Library / Framework	Primary Function within the System
Pandas	The Pandas serve as the primary data manipulation engine. It will load millions of rows from the raw files, resolve inconsistent column names, and dynamically map aliases to prepare a clean, structured data feed for the network controller.
numpy	The numpy will handle all high-speed, array-based mathematical operations. It will execute the fast calculations required for tuning the algorithm's thresholds and will strictly enforce a random seed to guarantee that future researchers can replicate the exact same mathematical results.
scikit-learn	The scikit-learn acts as the instructor and core machine learning engine. It will provide the mathematical architecture for both the Isolation Forest and Random Forest algorithms, systematically scale feature values to prevent processing overloads, and generate the necessary evaluation metrics.
imblearn	The imblearn will be utilized to mathematically balance the dataset by synthetically generating examples of rare attack types, ensuring the models do not ignore minority classes.

matplotlib & seaborn	The matplotlib & seaborn will be used to render visual performance charts and evaluation curves, allowing for a clear analysis of the system's detection accuracy.
Jinja	The Jina is the primary framework to construct a local graphical dashboard, enabling network administrators to monitor live traffic evaluations and system decisions in real-time.

Table 2. Frameworks & Libraries

Data Structures

The efficiency of the mitigation system depends heavily on the careful selection of data structures that allow the machine learning algorithms to execute within strict latency constraints. The system will rely on a single foundational data structure: the pandas DataFrame.

A DataFrame is a highly efficient, two-dimensional tabular data structure designed to handle millions of rows of data in memory simultaneously. In this system, the DataFrame will be used to explicitly construct and enforce a strict 24 feature matrix. When raw data is loaded, the DataFrame will act as an automated filter, systematically dropping heavy, unnecessary data columns (such as the total length of forward and backward packets) and retaining only the 18 specific network characteristics required by the system. This exact 18-column structure is engineered to perfectly mirror the chronological data extraction order of the Ryu SDN controller. By enforcing this rigid structure, the DataFrame ensures that the live data moving from the network switches directly matches the exact shape the machine learning models expect, preventing data-type errors and allowing for rapid, real-time detection.

Dataset Utilization

The algorithm will utilize the collected dataset to train the models before they are deployed to the active network controller. To ensure that the models learn correctly and can be verified without bias, the dataset will be strictly divided into three distinct subsets: an initial training set, a validation set used strictly for tuning the algorithm, and a final testing set.

Practical Challenges and Algorithmic Solutions Implementing machine learning in an active network presents specific practical challenges, which will be addressed using targeted techniques:

- Defining the exact boundary for an anomaly: The Isolation Forest will not arbitrarily guess the threshold. Instead, it will use a mathematical formula to find the optimal balance between true attack detections and false alarms. Crucially, a specific validation subset will be used exclusively for this tuning to avoid environmental bias from the standard dataset.

- Imbalanced attack data: Network data often contains an overwhelming majority of normal packets but very few examples of specific attacks. To prevent the models from ignoring rare attacks, the system will synthetically generate examples of minority classes. Furthermore, the Random Forest classifier will be structurally configured to mathematically force the algorithm to treat all attack types equally during the decision-making process.

- Preventing false bans: The Random Forest will utilize a probability-based confidence gate. If the algorithm's prediction fails to meet a predefined confidence threshold, it will output an "Uncertain" status rather than issuing a network drop rule, thereby prioritizing legitimate user traffic.

Code Snippets

To illustrate the structure of the system, Figure 3 presents the UML Class Diagram. This diagram maps the relationships between the Ryu Controller, the TrafficWatcher module, and the MLEngine, paving the way for the specific class implementations detailed in the code snippets below.

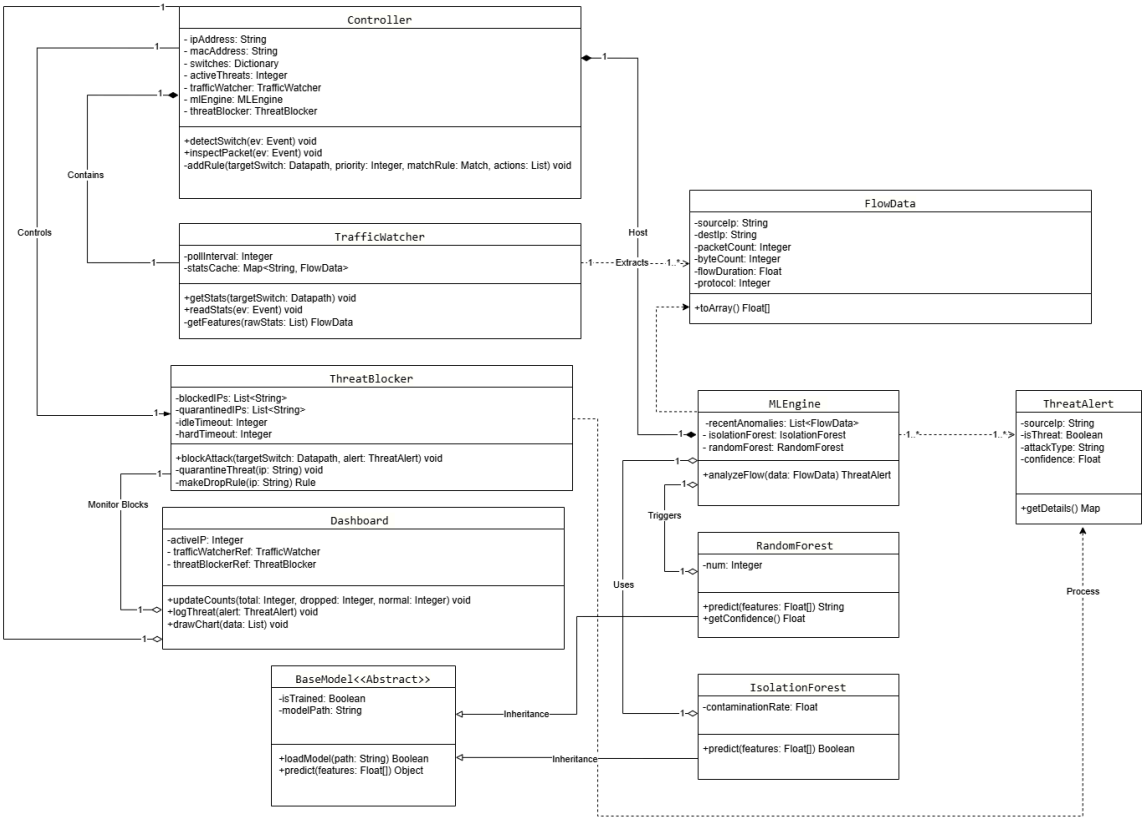

Figure 3. Class Diagram illustrating the Object-Oriented Architecture of the Mitigation System

The system's implementation will rely heavily on precise configuration and structured data handling during the training phase. Below are the code snippets extracted from the proposed implementation, demonstrating the initialization and setup protocols for both the primary and secondary machine learning models:

```
class IsolationForestDetector:

    def __init__(self, output_dir="if_results"):
        self.output_dir = output_dir
        self._create_output_directories()

        self.scaler         = RobustScaler()
        self.le_protocol    = LabelEncoder()
        self.model          = None
        self.threshold      = None

        self.training_log = {
            "start_time":        None,
            "end_time":          None,
            "dataset_info":      {},
            "preprocessing_info":{},
            "training_params":   {},
            "epoch_metrics":     [],
            "final_metrics":     {}
        }

        print("=" * 70)
        print("ISOLATION FOREST DETECTOR")
        print("Dataset : CIC-IDS / CIC-DDoS")
        print("=" * 70)
```

Initialization of the Primary Isolation Forest Detector

This snippet will illustrate the initial configuration of the primary anomaly detection engine. The system will prepare the necessary data scaling tools and establish a structured logging framework. This framework will be utilized to systematically record training parameters and evaluation metrics throughout the monitoring process.

```
def __init__(self, output_dir="rf_classifier_results"):
        self.output_dir      = output_dir
        self._create_output_directories()

        self.scaler          = RobustScaler()
        self.le_protocol     = LabelEncoder()
        self.label_encoder   = LabelEncoder()
        self.model           = None

        self.training_log = {
            "start_time":         None,
            "end_time":           None,
            "dataset_info":       {},
            "preprocessing_info": {},
            "training_params":    {},
            "epoch_metrics":      [],
            "final_metrics":      {},
        }
```

Initialization of the Secondary Random Forest Classifier

This snippet will illustrate the configuration phase of the secondary verification model. The system will set up the essential label encoders to properly format the specific attack categories and will initialize the data tracking structures required to accurately log the multi-class classification results.

Complexity Analysis

The proposed system operates across two distinct phases. The first is the offline training phase, where both machine learning models are built and prepared on the dataset before the system is deployed inside the Mininet and Ryu environment. The second is the runtime inference phase, where the already-trained models are loaded into the Ryu controller and used to evaluate live network traffic. These two phases carry different computational costs and must be analyzed separately.

Training Phase

The training phase prepares the machine learning models that will later analyze network traffic and detect potential threats. During this stage, historical network traffic data is used so that the models can learn patterns associated with normal behavior and malicious activity. The proposed system employs a two-stage cascaded architecture, where each model is trained with a specific objective to improve detection performance while maintaining computational efficiency.

In the first stage, Isolation Forest is trained to learn the statistical characteristics of normal network traffic and detect deviations that may indicate anomalies. In the second stage, Random Forest is trained using labeled attack data to classify and identify specific types of malicious traffic.

Isolation Forest Training

Isolation Forest is trained exclusively on BENIGN traffic samples from the dataset, consistent with its unsupervised nature. Rather than learning attack signatures, it learns the statistical boundaries of normal network behavior so that any significant deviation from that boundary can be flagged as a potential anomaly at runtime.

Isolation Forest operates by constructing a collection of isolation trees, where each isolation tree is built by recursively partitioning a random subsample of the dataset rather than the full dataset, using randomly selected features and split values. This subsampling behavior is a deliberate design decision grounded in the observation that anomalies are few and statistically distinct from normal samples: they can be isolated in very few splits regardless of how large the full dataset is, making the use of the entire dataset unnecessary and computationally wasteful. The training time complexity of Isolation Forest is:

$$O(t \bullet \psi \bullet log\, \psi)$$

where t is the number of isolation trees in the ensemble and ψ (psi) is the fixed subsample size used to build each isolation tree. This formula is derived as follows: each isolation tree partitions ψ subsampled points into a binary tree of maximum depth log ψ, since at each level the data is split approximately in half. At each level, partitioning the ψ points costs $O(\psi)$. Therefore, building one isolation tree costs $O(t \bullet \psi \bullet log\, \psi)$, and multiplying across all t trees gives the total training cost of $O(t \bullet \psi \bullet log\ \psi)$.

Critically, this complexity is expressed in terms of ψ rather than n (the full dataset size) because Isolation Forest never trains on the full dataset per isolation tree. In practice, the default values of t = 100 isolation trees and ψ = 256 points are sufficient for convergence.

This means the training cost is effectively constant and does not grow as n increases. Whether the dataset contains 100,000 or 10,000,000 records, the training time of the Isolation Forest remains unchanged, since each isolation tree always subsamples exactly ψ = 256 points.

Random Forest Training

Random Forest is trained exclusively on attack-labeled samples from the dataset, with all BENIGN rows excluded from training. This design is consistent with its role as Phase 2 of the cascaded architecture: it only needs to classify traffic that has already been identified as anomalous by the Isolation Forest, and therefore does not need to learn the characteristics of normal traffic.

Unlike Isolation Forest, Random Forest references the full training dataset n spread across bootstrapped samples, because it must learn precise decision boundaries between known attack classes and legitimate traffic. The training time complexity is:

$$O(k \bullet n \bullet m \bullet \log n)$$

where k is the number of decision trees, n is the number of attack training samples, and m is the number of features evaluated at each split node. This formula is derived as follows: a single decision tree on n samples grows to a maximum depth of $O(\log n)$, assuming reasonably balanced splits. At each node, m randomly selected features are evaluated, and evaluating one feature across the samples at that node costs $O(n)$. Therefore, building one decision tree costs $O(n \bullet m \bullet \log n)$, and multiplying across k trees gives the total training complexity of $O(k \bullet n \bullet m \bullet \log n)$.

Runtime Inference Phase

The runtime inference phase represents the actual complexity of the system as it operates live inside the Ryu SDN Controller. At this stage, both models are already trained and loaded into memory. The system evaluates incoming network flow records in real time based solely on prediction operations, not training.

The Ryu controller continuously polls OpenFlow switches for flow statistics and extracts a feature vector per flow record for Isolation Forest evaluation. The inference time complexity of Isolation Forest for a single incoming flow record is:

$$O(t \bullet \log \psi)$$

This is derived from traversing t isolation trees, where each isolation tree has a maximum depth of $\log \psi$. At each node, only a single feature comparison is performed. Traversing one isolation tree costs $O(\log \psi)$, and traversing all t isolation trees costs $O(t \bullet \log \psi)$. Because both t and ψ are fixed constants determined at training time, this inference cost is effectively constant per flow record and does not grow with the volume of live network traffic. This ensures that the frontline detection engine introduces no measurable latency to the control plane even as traffic volume increases.

For flows flagged as anomalous by Isolation Forest, a feature vector is constructed and passed to the Random Forest classifier. The inference time complexity of Random Forest for a single flagged flow is:

$$O(k \bullet log\ n)$$

This is derived from traversing k decision trees, each of maximum depth $O(log\ n)$, where each node involves only a single feature comparison. Since Random Forest is invoked only when Isolation Forest has already flagged a flow as suspicious, it is activated for only a small subset of total traffic rather than every flow record. This preserves the low-latency profile required for real-time SDN mitigation.

The overall runtime pipeline of the system for each cycle can be described as follows:

1. **Step 1** - Isolation Forest evaluates all incoming flow records at $O(t \bullet log\ \psi)$ per flow, which is approximately constant.
2. **Step 2** - Flagged flows only are forwarded to Random Forest for attack classification at $O(k \bullet log\ n)$ per flagged flow.
3. **Step 3** - Confirmed attack flows trigger a FlowMod directive at $O(1)$message dispatch via the Ryu controller.

This design ensures that the computationally heavier Random Forest classifier is never applied to the full stream of network traffic, which preserves the low-latency profile required for real-time SDN mitigation.

Space Complexity

Memory efficiency is a critical operational requirement for a system deployed within an SDN controller, as excessive memory consumption risks saturating the control plane and degrading its ability to manage network-wide forwarding decisions.

The Isolation Forest model stores all of its isolation trees in memory after training. Each isolation tree holds at most 2ψ - 1 nodes, since a binary tree built from ψ subsampled points has at most ψ leaf nodes and ψ - 1 internal nodes. The total memory footprint of the entire Isolation Forest ensemble is therefore:

$$O(t \bullet \psi)$$

Because both t and ψ are small fixed constants, the memory occupied by the Isolation Forest does not grow with the size of the dataset or the volume of live traffic. The model is loaded into memory once at system startup and remains there unchanged throughout the session.

The Random Forest model similarly stores all of its decision trees in memory after training. The total memory footprint depends on the number of trees k and the number of training samples n used to build them, giving a space complexity of:

Like the Isolation Forest, the Random Forest is loaded once at system startup as a static serialized file and does not change during live operation. In addition, the system maintains a rolling buffer of flow statistics polled from the OpenFlow switches each cycle. This buffer holds O(f) records, where f is the number of active flows at any given moment, and does not grow indefinitely as records from the previous polling cycle are replaced by newer ones on each update.

Scalability and Practical Limitations

The proposed system is designed with lightweight runtime operation as a primary constraint. Because Isolation Forest's training complexity is bounded by the fixed subsample size ψ rather than the full dataset n, the model scales efficiently even as the dataset grows or as the system is retrained on larger traffic captures in future studies.

Adding more training records does not increase training time, since each isolation tree always subsamples exactly $\psi = 256$ points regardless of how large n becomes. This is a significant scalability advantage for large-scale network security applications.

As the network topology grows, the number of active flows f polled from OpenFlow switches also increases with the number of hosts and switches. In the tree topology used in this study, f remains manageable within the Mininet virtual environment. However, scaling to enterprise-grade topologies with hundreds of switches would increase the per-cycle polling overhead, and flow aggregation strategies or adjusted polling intervals may be necessary to maintain real-time performance at larger scales.

Furthermore, the system's mitigation capacity is bounded by the processing and memory limitations of the host machine running the Ubuntu 22.04 virtual environment. While the cascaded architecture is deliberately lightweight at inference time, total mitigation cannot be guaranteed if traffic volume surpasses the virtual machine's processing capacity.

Since both models are trained offline on a fixed dataset, their ability to detect attack patterns that were not present during training may weaken over time without periodic retraining on newer traffic captures. Overall, the proposed Isolation Forest and Random Forest architecture maintains a deliberate balance between detection accuracy, response latency, and controller resource overhead, making it a feasible and scalable solution for anomaly-based DDoS mitigation in Software-Defined Networks.

System Development Methodology

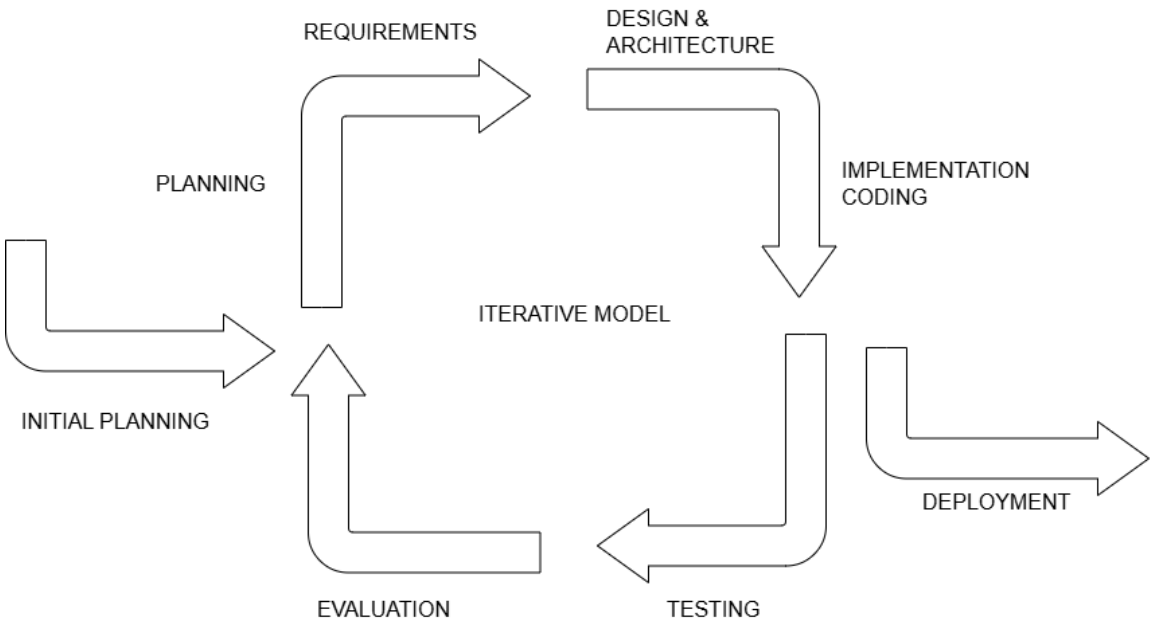

Figure 5. Iterative Methodology

The Iterative methodology consists of several key phases, namely requirements analysis, iteration planning, system design, implementation, testing and evaluation, and system deployment, as illustrated in **Figure 5.** The process begins with the overall project objective, which is divided into smaller, functional components that undergo continuous development cycles. Each iteration focuses on developing and refining specific system features, such as establishing the Mininet network baseline or integrating machine learning models.

During each cycle, selected components are planned, designed, implemented, and tested to produce a working prototype. The system is then evaluated based on performance metrics such as detection accuracy and Ryu controller CPU utilization. Based on the results, necessary refinements are made before proceeding to the next

iteration. This cycle continues until the complete cascaded architecture is fully developed and optimized.

Iteration Planning. During this phase, the overall project is broken down into specific, manageable goals for the current cycle. The team decides which specific component will be developed, such as the initial Mininet topology, the traffic feature extraction script, or the integration of the Isolation Forest algorithm.

Design and Architecture. This phase focuses on the logical mapping of the iteration's features. The team designs the network topology (hosts, switches, and controller links) and the internal logic of the machine learning pipeline. It ensures that the data flow between the Ryu controller and the virtual switches is clearly defined before any coding begins.

Implementation. The planned designs are translated into executable Python scripts. The lead programmer develops the Ryu application modules, while the rest of the team configures the Mininet environment and prepares the machine learning libraries to ensure the functional prototype is ready for execution.

Testing and Evaluation. Once the iteration is coded, the prototype is subjected to rigorous experimental testing. Volumetric DDoS attacks are simulated to measure the system's performance. The researchers evaluate the results based on specific metrics, primarily focusing on detection accuracy and the CPU utilization of the Ryu controller to see if the current iteration meets the project's performance goals. These metrics and the status of the network are fed into a web-based dashboard. This dashboard provides real-time visualization of the traffic flows, resource consumption graphs, and attack alerts, allowing the researchers to visually confirm if the current iteration meets the project's performance goals.

System Deployment. Once all iterations are complete and the cascaded machine learning framework has been optimized through repeated cycles, the final system is deployed. This version represents the fully verified security application,

capable of protecting the SDN controller from resource exhaustion during active DDoS floods.

Statistical Treatment of Data

The data collected from both the simulated network environment and the expert respondents will be analyzed using appropriate descriptive and inferential statistical tools to ensure valid interpretation of the results [2]. The statistical treatments are categorized into the evaluation of the machine learning algorithms, which relies on objective performance metrics, and the analysis of the software quality assessment, which utilizes central tendency measures for expert feedback.

Population and Sampling

The population for this study is categorized into digital and human components. For the human respondents, the study utilizes purposive sampling, a non-probability technique in which participants are selected based on specific professional criteria directly relevant to the research objectives [49].

The target human population consists of a maximum of fifteen (15) professionals based in Laguna and its neighboring provinces, selected due to the high concentration of industrial and academic institutions utilizing localized network infrastructures in the area. Meanwhile, the digital population consists of the total volume of network flow entries derived from the SNT DDoS Detection Dataset and the DDoS Attacks Dataset for Software Defined Network, ensuring the system is evaluated against a statistically significant volume of both legitimate and adversarial traffic patterns.

Respondents must have a minimum of 3 to 5 years of professional experience in positions such as Network Administrator, Network Engineer, or IT Administrator, with direct expertise in Software-Defined Networking (SDN) management or cybersecurity.

This includes ten (10) Network Administrators, two (2) IT Experts, and three (3) Academic Validators.

This extensive coverage guarantees evaluative viewpoints from operational, architectural, and theoretical angles, enabling a critical assessment of the cascaded anomaly detection pipeline (Isolation Forest and Random Forest), the resource-saving efficacy of the Ryu controller, and the automated "Decision-to-Action" mitigation protocol.

The selected professionals serve a dual role in the validation phase. They first act as technical evaluators, verifying the soundness of the virtualized Mininet testbed and the accuracy of the algorithms in classifying incoming OpenFlow traffic.

The same group subsequently evaluates the web-based visualization dashboard and the system's overall operational viability using the ISO 25010 software quality assessment and semi-structured interviews, enabling a comprehensive assessment of both internal security logic and practical usability within a single structured session.

Category of Respondents	**Number of Respondents**
Network Administrators	10
IT Experts	2
Academic Validator	3
Total	15

Table 3. Population & Sampling

Evaluation and Scoring

The evaluation phase of this research is designed to validate the effectiveness of the cascaded machine learning model in a live Software-Defined Networking (SDN) environment. The assessment follows a two-tiered approach comprising a technical performance analysis based on simulated network traffic and a qualitative software quality audit based on international standards.

The primary objective of the technical evaluation is to determine the accuracy and efficiency of the proposed dual-layer detection system. Using the Ryu controller and Mininet emulator, the system's components are evaluated using datasets specifically aligned with their roles in the cascade.

The initial anomaly detection layer, utilizing Isolation Forest, is evaluated using the SNT (Simulated Network Traffic Using Mininet and Ryu) DDoS Detection Dataset to ensure high sensitivity to outliers within the specific emulation environment.

The subsequent verification layer, utilizing Random Forest, is tested against the DDOS ATTACKS DATASET FOR SOFTWARE DEFINED NETWORK to validate its ability to precisely classify specialized SDN attack vectors. Detection fidelity is measured through precision and recall, where high recall is prioritized in the Isolation Forest tier to ensure no potential threats bypass the filter, while high precision is required in the Random Forest tier to prevent erroneously blocking legitimate users.

Additionally, the study measures mitigation latency, or "Decision-to-Action" speed, which tracks the time interval between the initial detection of a statistical outlier and the successful installation of a FlowMod entry in the OpenFlow switch. To address the problem of CPU exhaustion, controller resource utilization is monitored to ensure the lightweight nature of the unsupervised model preserves the network's central processing capabilities.

In addition to technical metrics, the system is evaluated against the ISO 25010 Software Quality Model to ensure its viability as a network security tool. This assessment is conducted by IT professionals and network administrators through a validated research instrument focusing on three specific characteristics. First, Functional Suitability evaluates how completely the system automates the mitigation process and achieves autonomous threat isolation without human oversight. Second, Performance Efficiency measures the system's responsiveness and resource consumption levels under varying network loads, specifically looking for the mitigation of processing bottlenecks.

Finally, the Security characteristic assesses the system's ability to protect the SDN control plane from exhaustion and maintain the integrity of the data plane during an active DDoS event.

The qualitative feedback from the respondents is quantified using a four-point Likert scale. By removing a neutral midpoint, the scoring system requires evaluators to provide a clear indication of the system's effectiveness. The scores gathered are processed to find the weighted mean, which is then interpreted according to established ranges to determine the system's overall success. This involves comparing the qualitative ratings with the technical accuracy and latency results to ensure the final solution is both efficient and practically viable.

Weighted Mean Range	**Assigned Value**	**Assessment**	**Verbal Interpretation**
3.51 - 4.00	4	Strongly Agree	Excellent
2.51 - 3.50	3	Agree	Satisfactory

1.51 - 2.50	2	Disagree	Unsatisfactory
1.00 - 1.50	1	Strongly Disagree	Poor

Table 4. Four-Point Likert Scale Method

Ethical Considerations

This research is conducted with strict adherence to established ethical guidelines in both social science data collection and cybersecurity testing. The ethical considerations for this study are categorized into two main themes:

Protection of Data Privacy

The respondents involved in the ISO 25010 survey and semi-structured interviews, the study strictly complies with the provisions of Republic Act No. 10173, otherwise known as the Data Privacy Act of 2012. Prior to any data collection, all respondents will be provided with an informed consent form detailing the study's purpose, the nature of their involvement, and how their data will be utilized. It will be emphasized that participation is strictly voluntary, and participants reserve the right to withdraw from the study at any time without penalty or consequence. To ensure confidentiality and anonymity, all responses will be anonymized to protect the identities of the IT professionals, network experts, and their respective organizations. Any identifying information, such as names or specific company affiliations, will be strictly coded or redacted in the final manuscript. Furthermore, all qualitative transcripts and survey results will be stored securely in encrypted, password-protected local drives

accessible only to the researchers. These files will be properly and permanently disposed of after the completion and successful defense of the study.

Safe Execution of Simulations

The technical phase of this study involves orchestrating malicious Distributed Denial-of-Service (DDoS) traffic, making ethical technical execution paramount to ensure that no real-world networks are harmed. To achieve this, the simulated cyberattacks will be strictly confined within a localized, virtualized sandbox environment utilizing the Mininet emulator and the Ryu SDN controller on an isolated Ubuntu host machine.

Because the system is exclusively implemented and tested within this standalone virtual sandbox, the researchers guarantee that it is physically isolated from, and cannot leak malicious traffic into, any external local area networks (LANs), personal hardware, or public internet infrastructure.

The study utilizes two specific open-source, academically approved datasets to train and test the cascaded machine learning models. The frontline unsupervised model (Isolation Forest) utilizes the Simulated Network Traffic Using Mininet and Ryu (SNT) DDoS Detection Dataset, while the secondary supervised validation layer (Random Forest) utilizes the DDoS Attacks Dataset for Software Defined Networks. The researchers commit to using these datasets solely for the purpose of anomaly detection and mitigation research in strict adherence to their licensing terms. Neither dataset contains real-world, unprotected Personally Identifiable Information (PII), thereby ensuring that no actual user data is compromised or exposed during the algorithmic training and simulation phases.

Literature Cited

[1] VMware, "What is Software-Defined Networking (SDN)? | VMware Glossary," VMware, 2024. [Online]. Available: https://www.vmware.com/topics/software-defined-networking

[2] S. Peng, J. Tian, X. Zheng, S. Chen, and Z. Shu, "DDoS Defense Strategy Based on Blockchain and Unsupervised Learning Techniques in SDN," Future Internet, vol. 17, no. 8, pp. 367–367, Aug. 2025, doi: https://doi.org/10.3390/fi17080367.

[3] O. Yoachimik and J. Pacheco, "Cloudflare's 2025 Q3 DDoS threat report -- including Aisuru, the apex of botnets," The Cloudflare Blog, Dec. 03, 2025. [Online]. Available: https://blog.cloudflare.com/ddos-threat-report-2025-q3/.

[4] R. Ma, Q. Wang, X. Bu, and X. Chen, "Real-Time Detection of DDoS Attacks Based on Random Forest in SDN," Applied Sciences, vol. 13, no. 13, p. 7872, Jul. 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/13/7872

[5] T. S. Chu, W. Si, Simeon Simoff, and Q. V. Nguyen, "A Machine Learning Classification Model Using Random Forest for Detecting DDoS Attacks," 2022 International Symposium on Networks, Computers and Communications (ISNCC), pp. 1–7, Jul. 2022, doi: https://doi.org/10.1109/isncc55209.2022.9851797.

[6] I. A. Mahar et al., "A hybrid machine learning approach for detecting DDoS attacks in software-defined networks," Scientific Reports, Jan. 2026. [Online]. Available: https://www.nature.com/articles/s41598-026-35458-w

[7] I. Ko, D. Chambers, and E. Barrett, "Unsupervised learning with hierarchical feature selection for DDoS mitigation within the ISP domain," ETRI Journal, vol. 41, no. 5, pp. 574–584, Sep. 2020. [Online]. Available: https://onlinelibrary.wiley.com/doi/full/10.4218/etrij.2019-0109

[8] M. Najafimehr, S. Zarifzadeh, and S. Mostafavi, "DDoS attacks and machine-learning-based detection methods: A survey and taxonomy," Engineering Reports, vol. 5, no. 12, May 2023. [Online]. Available: https://onlinelibrary.wiley.com/doi/full/10.1002/eng2.12697

[9] K. Nisar, I. Welch, R. Hassan, A. H. Sodhro, and S. Pirbhulal, "A Survey on the Architecture, Application, and Security of Software Defined Networking," Internet of Things, vol. 12, p. 100289, Sep. 2020, [Online] Available: https://doi.org/10.1016/j.iot.2020.100289.

[10] M. S. Lakshmi, G. Rajavikram, V. Dattatreya, B. S. Jyothi, S. Patil, and M. Bhavsingh, "Evaluating the Isolation Forest Method for Anomaly Detection in Software-Defined Networking Security," Journal of Electrical Systems, vol. 19, no.4,pp.279–297,2023.[Online].Available:https://pdfs.semanticscholar.org/d42f/e4e5e1f8ea0f547014bb8e1a124445c11b76.pdf

[11] S. K. Pattanayak, M. Bhoyar, and T. Adimulam, "Unsupervised Learning for Anomaly Detection in Cybersecurity," International Journal of Research Culture Society, vol. 8, no. 12, pp. 56–64, Dec. 2024. [Online]. Available: https://ijrcs.org/wp-content/uploads/IJRCS202412011-min.pdf

[12] A. A. Alashhab et al., "Enhancing DDoS Attack Detection and Mitigation in SDN using an Ensemble Online Machine Learning Model," IEEE Access, Jan. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10489935

[13] S. Sharan, S. Mani, and S. Kumar, "A Comprehensive Review of DDoS Detection and Mitigation in SDN Environments: Machine Learning, Deep Learning, and Federated Learning Perspectives," Electronics, vol. 14, no. 1, p. 222,Jan.2025.[Online].Available:https://www.mdpi.com/2079-9292/14/21/4222

[14] J. Shaikh et al., "Advancing DDoS attack detection with hybrid deep learning: integrating convolutional neural networks, PCA, and vision transformers," International Journal on Smart Sensing and Intelligent Systems, vol. 17, no. 1, Apr.2024.[Online].Available:https://sciendo.com/2/v2/download/article/10.2478/ijssis-2024-0040.pdf

[15] M. L. Ram, C. A. G. Gabi, J. C. Agullo, R. D. Clemente, R. D. Malangsa, and D. S. Lintao, "Unsupervised Anomaly Detection in IoT Attacks Using Isolation Forest on the Kitsune Dataset," in Lecture Notes in Networks and Systems, Jan. 2026, pp. 152–162. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-10827-2_12

[16] P. Karthika and K. Arockiasamy, "Simulation of SDN in mininet and detection of DDoS attack using machine learning," Bulletin of Electrical Engineering and Informatics, vol. 12, no. 3, pp. 1655–1662, Jun. 2023.

[17] R. Sanjeetha, A. Kanavalli, A. Gupta, A. Pattanaik, and S. Agarwal, "Real-time DDoS detection and mitigation in software defined networks using machine learning techniques," IEEE Access, vol. 9, pp. 38061–38071, 2021.

[18] M. Francis, M. Aquino, and M. Noroña, "Enhancing Cyber Security in the Philippine Academe: A Risk-Based IT Project Assessment Approach," IEOM Society, 2021. [Online]. Available: https://www.ieomsociety.org/singapore2021/papers/878.pdf

[19] A. G. A. Abd-Allah et al., "DDoS Mitigation Using Machine Learning in Software-Defined Networks," Journal of Computer Science, vol. 21, no. 4, pp. 940–960, Apr. 2025.

[20] M. Nowsin, I. Hwang, M. S. Raza, and M. S. Ab-Rahman, "A Qualitative and Comparative Performance Assessment of Logically Centralized SDN Controllers via Mininet Emulator," Computers, vol. 13, no. 4, p. 85, Mar. 2024. [Online]. Available: https://www.mdpi.com/2073-431X/13/4/85

[21] L. Borgianni, C. Bua, E. Coli, D. Adami, and S. Giordano, "On the accuracy of mininet: Comparing network emulation and physical testbeds for Distributed Systems," in 2025 17th International Congress on Ultra Modern Telecommunications and Control Systems, Nov. 2025, pp. 158–163. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11268697/

[22] A. Almaini et al., "Integrating reality: A hybrid sdn testbed for enhanced realism in Edge Computing simulations," in 2024 International Conference on Ubiquitous Networking (UNet), Jun. 2024, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10794738

[23] J. K. Candelario, "A quantitative correlational study of assessing the effectiveness of Random Forest in reducing false positives in intrusion detection systems for enterprise networks," ResearchGate, Jan. 2025. [Online]. Available: https://www.researchgate.net/publication/397063522.

[24] H. A. Butt, A. Harthy, M. A. Shah, M. Hussain, R. Amin, and M. U. Rehman, "Enhanced DDoS Detection Using Advanced Machine Learning and Ensemble Techniques in Software Defined Networking," Computers, Materials & Continua, vol. 81, no. 2, pp. 3003–3031, Jan. 2024.

[25] A. Hamarshe, H. I. Ashqar, and M. Hamarsheh, "Detection of DDoS attacks in software defined networking using machine learning models," arXiv preprint arXiv:2303.06513, Mar. 2023.

[26] E. Horsanali, Y. Yigit, A. Karameseoglu, G. Secinti, and B. Canberk, "Network-aware AutoML framework for software-defined sensor networks," arXiv preprint arXiv:2310.12914, Oct. 2023.

[27] M. E. Haque et al., "Optimizing DDoS detection in SDNs through machine learning models," in Proc. IEEE 16th Int. Conf. Comput. Intell. Commun. Netw. (CICN), 2024.

[28] G. Chen, “Cyber-physical energy system security and resilience enhancement with scalable and high-fidelity emulation evaluation,” Ph.D. dissertation, Dept. Comput. Sci., Illinois Inst. Technol., Chicago, IL, USA, 2024.

[29] Cloudflare, "Global DDoS Threat Report: The Rise of Ransom DDoS and Volumetric Attacks," Cloudflare Security Research, 2023. [Online]. Available: https://blog.cloudflare.com/ddos-threat-report.

[30] W. Song, Z. Alomari, X. Zhang, and B. Xie, “Mitigating Data Imbalance in DDoS Detection for SDN Through Machine Learning Methods,” in 2025 1st International Conference on Computational Intelligence Approaches and Applications (ICCIAA), Vancouver, BC, Canada, 2025, pp. 1–6.

[31] Sumit Badotra et al., “A DDoS Vulnerability Analysis System against Distributed SDN Controllers in a Cloud Computing Environment,” Electronics, vol. 11, no. 19, pp. 3120–3120, Sep. 2022, [Online]. Available: A DDoS Vulnerability Analysis System against Distributed SDN Controllers in a Cloud Computing Environment

[32] E. P. Estupiñán Cuesta, J. C. Martínez Quintero, and J. D. Avilés Palma, "DDoS Attacks Detection in SDN Through Network Traffic Feature Selection and Machine Learning Models," Telecom, vol. 6, no. 3, p. 69, Sep. 2025, doi: 10.3390/telecom6030069.

[33] A. J. Ibrahim, S. R. Répás, and N. Bektaş, "Feature-Optimized Machine Learning Approaches for Enhanced DDoS Attack Detection and Mitigation," Computers, vol. 14, no. 11, p. 472, Nov. 2025, doi: 10.3390/computers14110472.

[34] N. F. Rozam, H. Jati, and E. Marpanaji, "Comparative Study of Machine Learning Algorithms for DDoS Attack Detection in SDN Networks: A Carbon Emission Analysis with Hyperparameter Optimization Using Bayesian Optimization," Int. J. Adv. Sci. Eng. Inf. Technol., vol. 15, no. 2, 2025.

[35] A. C. Fatiyah, A. Abbas, P. E. Setiasabda, W.-B. Hsieh, J.-S. Leu, and S.-J. Chen, "Machine Learning-Based Real-Time Detection and Mitigation of DoS Attacks in SDN-Based 5G Network," Electronics, vol. 15, no. 5, p. 1005, 2026, doi: 10.3390/electronics15051005.

[36] A. Alsufyani, B. Alotaibi, and S. Alajmani, "Hybrid Deep Learning Approach for Enhanced Detection and Mitigation of DDoS Attack in SDN Networks," Int. J. Netw. Secur. Its Appl., vol. 16, no. 6, Nov. 2024.

[37] A. O. Isaiah, A. Abdullah, N. Samian, and Z. M. Hanapi, "Flow Table Overflow Attacks in a Software-Defined Network (SDN): A Systematic Review," IAENG Int. J. Comput. Sci., vol. 51, no. 9, 2024.

[38] H. Zaheed et al., "On the (in)Security of the Control Plane of SDN Architecture: A Survey," IEEE Access, vol. 11, pp. 91200-91225, 2023.

[39] W. Xu et al., "Measuring the Consistency Between Data and Control Plane in SDN," IEEE/ACM Transactions on Networking, vol. 31, no. 2, pp. 642-655, April 2023.

[40] M. A. Saleem et al., "Enhancing Software-Defined Networks with Intelligent Controllers to Improve First Packet Processing Period," Electronics, vol. 12, no. 3, p. 600, 2023.

[41] R. Chu et al., "Concept Drift Analysis Based on Isolation Forest for Effectively Detecting Network Attack in IoT Scenarios," IEEE Access, vol. 14, pp. 1120-1135, 2026.

[42] S. G. K. Patra et al., "Building a Mininet-based Fiber Optic Simulator," 2021 IEEE International Conference on Computing, Power and Communication Technologies (GUCON), pp. 1-6, 2021.

[43] J. Shin et al., "Delay of Flow Insertion and Flow Modification, as Observed from the Data Plane," IEEE Journal on Selected Areas in Communications, vol. 39, no. 10, pp. 3110-3125, 2021.

[44] M. Roopak, G. Tian, and J. Chambers, “Multi-Objective based Feature Selection for DDoS Attack Detection in IoT Network,” IET Networks, Jan. 2020.[Online].Available:https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/iet-net.2018.5206

[45] M. Shariq et al., "Hybrid Machine Learning Models: Combining Strengths of Supervised and Unsupervised Learning Approaches," 2024 IEEE International Conference on Cybersecurity and AI, pp. 45-51, 2025.

[46] T. J. Choi et al., "A Hybrid Unsupervised Machine Learning Model for Anomaly Detection in Industrial Control Systems," MDPI Sensors, vol. 24, no. 4, p. 1118, 2024.

[47] CrowdStrike, "What is a Zero-Day Exploit?," CrowdStrike Cybersecurity 101, 2025. [Online]. Available: https://www.crowdstrike.com/cybersecurity-101/cyberattacks/zero-day-exploit/.

[48]

[49] O. Tajik, J. Golzar, and S. Noor, "Purposive Sampling," International Journal of Education & Language Studies, vol. 2, no. 2, pp. 1-13, Dec. 2024. [Online]. Available: https://doi.org/10.22034/ijels.2025.490681.1029

[50] T. Glad and L. Ljung, *Control Theory: Multivariable and Nonlinear Methods*. London: Taylor & Francis, 2000. [Online]. Available: https://doi.org/10.1201/9781315274737

[51] D. Moloja and V. Malele, "Software-Defined Networking powered by AI-driven Anomaly Detection," Journal of Cybersecurity Education, Research and Practice, vol. 2025, no. 1, Nov. 2025. [Online]. Available: https://digitalcommons.kennesaw.edu/jcerp/vol2025/iss1/24/

[52] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouët, "Anomaly Detection in Streams with Extreme Value Theory," in Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, 2017, pp. 1067–1075. [Online]. Available: https://dl.acm.org/doi/epdf/10.1145/3097983.3098144

[53] J. Kittler and F. Roli, Eds., Multiple Classifier Systems: Second International Workshop, MCS 2001, Cambridge, UK, July 2-4, 2001, Proceedings, vol. 2096. Berlin, Germany: Springer-Verlag, 2001. [Online]. Available: https://books.google.com.ph/books?hl=en&lr=&id=Ka52SwzchBwC&oi=fnd&pg=PR3&dq=Kittler%E2%80%99s+Theory+of+Multiple+Classifier+Systems&ots=rtVQKTm8Q6&sig=aDBt1nZCrSsjxBQGJgraUPoyPI0&redir_esc=y#v=onepage&q=Kittler%E2%80%99s%20Theory%20of%20Multiple%20Classifier%20Systems&f=false

[54] J. Chen, S. Zhang, and L. Wang, "Information Theory-based Feature Selection and Anomaly Detection for Network Security," in Proc. IEEE Int. Conf. Commun., 2023, pp. 1024-1030. [Online]. Available: https://ieeexplore.ieee.org/document/10059949/

[55] GeeksforGeeks, "What is Anomaly Detection?," Jan. 19, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/what-is-anomaly-detection/

[56] GeeksforGeeks, "Control Plane vs Data Plane," Jul. 20, 2023. [Online]. Available: https://www.geeksforgeeks.org/control-plane-vs-data-plane/

[57] GeeksforGeeks, "What is DDoS (Distributed Denial of Service)?," Jul. 12, 2025. [Online]. Available:

https://www.geeksforgeeks.org/computer-networks/what-is-ddosdistributed-denial-of-service/

[58] GeeksforGeeks, "Software Defined Networking (SDN) - A Brief Overview," Mar. 09, 2026. [Online]. Available: https://www.geeksforgeeks.org/software-defined-networking-sdn/

[59] GeeksforGeeks, "Semi-Supervised Learning in ML," 2024. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/ml-semi-supervised-learning/

[60] GeeksforGeeks, "What is Isolation Forest," Nov. 11, 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/what-is-isolation-forest/

[61] GeeksforGeeks, "Mininet Commands," Aug. 22, 2024. [Online]. Available: https://www.geeksforgeeks.org/linux-unix/mininet-commands/

[62] GeeksforGeeks, "OpenFlow vs NETCONF," Jul. 23, 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/openflow-vs-netconf/

[63] GeeksforGeeks, "Random Forest Classifier using Scikit-learn," Mar. 06, 2026. [Online]. Available: https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/

[64] GeeksforGeeks, "What is Unsupervised Learning," Dec. 10, 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/unsupervised-learning/

[65] GeeksforGeeks, "What is Zero-Day Attack?," 2024. [Online]. Available: https://www.geeksforgeeks.org/what-is-zero-day-attack/

[66] A. A. Wabi, I. Idris, O. M. Olaniyi, J. A. Ojeniyi, and O. S. Adebayo, "DDOS ATTACKS DATASET FOR SOFTWARE DEFINED NETWORK," Mendeley Data, V1, 2023. [Online]. Available: https://data.mendeley.com/datasets/x6vr3sdm75/1

[67] Aymen Alawdi "SNT (Simulated Network Traffic Using Mininet and Ryu) DDoS Detection Dataset," Mendeley Data, V1. [Online]. Available: https://data.mendeley.com/datasets/9hz6f62gtk/1

[68] E. P. Ghani, A. Sofwan, and M. Somantri, "AI-Driven Network Security: Detecting and Mitigating DDoS, Malware, and Backdoor Attacks with Isolation and Random Forest Algorithm," in 2025 International Conference on Smart

Computing, IoT and Machine Learning (SIML), 2025, [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11080951

[69] Z. Zheng, Y. Chen, L. Chen, G. Guo, Y. Fan, and X. Kong, "Signal-BNF: A Bayesian Network Fusing Approach to Predict Signal Peptides," *Journal of Biomedicine and Biotechnology*, vol. 2012, Art. no. 492174, Sep. 2012, doi: 10.1155/2012/492174.